Universidade Politécnica de Tomar
Polytechnic University

Escola Superior de Tecnologia de Tomar

Tarefa Laboratorial #03:

- **Divisão inteira e raiz quadrada**
- **Com arrays em Assembly 8086 (emu8086)**

Guilherme Carril Pereira nº27226

Daniel Filipe Batista Petinga nº27466

Relatório apresentado à Universidade Politécnica de Tomar no âmbito da unidade curricular Arquitetura de Computadores.

RESUMO

Este relatório apresenta a resolução, em Assembly 8086 no emulador emu8086, da Tarefa 3: a divisão inteira e a raiz quadrada com arrays, seguindo os algoritmos definidos na aula. Os números são pedidos ao utilizador e guardados em arrays, um algarismo por posição, e podem ser negativos e/ou ter parte decimal. As contas sobre os arrays usam apenas adição, subtração e multiplicação, algarismo a algarismo, e o programa mostra no ecrã cada operação parcial e, no fim, o resultado. A divisão de números reais resolve-se com a mesma divisão inteira, depois de igualar as casas decimais. Apresentam-se os algoritmos, o pseudocódigo, os fluxogramas, as instruções utilizadas e os testes; os códigos, comentados e sem comentários, estão nos anexos.

Palavras-chave: Assembly 8086; emu8086; arrays; divisão inteira; raiz quadrada; aritmética algarismo a algarismo

ABSTRACT

This report presents the solution, in 8086 Assembly on the emu8086 emulator, of Task 3: integer division and square root using arrays, following the algorithms defined in class. The numbers are read from the user and stored in arrays, one digit per position, and may be negative and/or have a decimal part. The arithmetic on the arrays uses only addition, subtraction and multiplication, digit by digit, and the program shows every partial operation on the screen and then the final result. Real-number division is solved with the same integer division, after equalising the decimal places. The algorithms, pseudocode, flowcharts, instructions used and tests are presented; the code, with and without comments, is in the appendices.

Keywords: 8086 Assembly; emu8086; arrays; integer division; square root; digit-by-digit arithmetic

ÍNDICE

TOC \t "estilo-titulo,1" AGRADECIMENTOS vi
RESUMO vii
ABSTRACT viii
ÍNDICE ix
INTRODUÇÃO 10
ALGORITMOS DEFINIDOS NA AULA 11
PSEUDOCÓDIGO 14
FLUXOGRAMAS 23
INSTRUÇÕES UTILIZADAS 41
TESTES E RESULTADOS 44
CONCLUSÃO 47
BIBLIOGRAFIA 48
ANEXOS 49

INTRODUÇÃO

A Tarefa 3 pede o código em Assembly x86 de 16 bits, no emulador emu8086, de dois algoritmos definidos na aula, ambos com arrays:

– Divisão inteira: pedir os valores ao utilizador, recebê-los em variáveis vetores, permitir números negativos e/ou reais, efetuar os cálculos apenas com adição, subtração e multiplicação em arrays, apresentar no ecrã as operações parciais passo a passo e o resultado final. A operação de divisão inteira resolve tanto as divisões inteiras como as reais.

– Raiz quadrada: pedir o valor ao utilizador, recebê-lo numa variável vetor, efetuar os cálculos apenas com adição, subtração e multiplicação em arrays, apresentar as operações parciais passo a passo e o resultado final.

Os dois programas guardam cada número num array de bytes, com um algarismo por posição, tal como se faz no papel. Um registo de 16 bits só guarda valores até 65535 (ou de −32768 a 32767, com sinal), mas com os números em arrays o limite deixa de ser o tamanho dos registos: as contas fazem-se algarismo a algarismo e cada algarismo, ou cada produto de dois algarismos, cabe num byte. Nos programas, o utilizador pode escrever até 12 algarismos por número e cada array tem 26 posições, o que chega para os zeros que se acrescentam ao igualar as casas decimais.

Nenhum dos programas usa a instrução div. As únicas operações sobre os arrays são a soma, a subtração e a multiplicação. As dezenas de cada produto contam-se subtraindo 10, também sem dividir.

O relatório apresenta, por esta ordem, os algoritmos da aula com exemplos, o pseudocódigo, os fluxogramas, as instruções utilizadas, os testes e a conclusão. Os códigos estão nos anexos, em versão comentada e sem comentários.

ALGORITMOS DEFINIDOS NA AULA

Divisão inteira

A divisão faz-se como no papel, algarismo a algarismo, a partir do algarismo mais significativo do dividendo:

– Número = número anterior × 10 + algarismo seguinte do dividendo.

– Verificar se o número é maior ou igual ao divisor. Se não for, o algarismo do quociente é 0.

– Se for, faz-se a tabuada do divisor (divisor × 1, × 2, × 3, …) até o produto passar o número, e usa-se o algarismo anterior.

– Resto = número − divisor × algarismo; este resto é o número anterior do passo seguinte.

– Quociente = quociente × 10 + algarismo calculado.

Exemplo da aula: 7398451078 ÷ 25. No 1.º passo, 7 ≥ 25 é falso (algarismo 0). No 2.º, o número é 7 × 10 + 3 = 73 ≥ 25; 25 × 1 = 25, 25 × 2 = 50 e 25 × 3 = 75 > 73, por isso usa-se 2; resto = 73 − 50 = 23 e quociente = 0 × 10 + 2 = 2. A tabela mostra todos os passos, tal como o programa os escreve.

Passo e número	Verificar e tabuada	Resto	Quociente
1: 7	7 ≥ 25: Não; algarismo 0	7	0×10+0 = 0
2: 7×10+3 = 73	73 ≥ 25: Sim; 25×1=25, 25×2=50, 25×3=75 > 73 → usar 2	73 − 50 = 23	0×10+2 = 2
3: 23×10+9 = 239	239 ≥ 25: Sim; 25×1=25, 25×2=50, 25×3=75, 25×4=100, 25×5=125, 25×6=150, 25×7=175, 25×8=200, 25×9=225	239 − 225 = 14	2×10+9 = 29
4: 14×10+8 = 148	148 ≥ 25: Sim; 25×1=25, 25×2=50, 25×3=75, 25×4=100, 25×5=125, 25×6=150 > 148 → usar 5	148 − 125 = 23	29×10+5 = 295
5: 23×10+4 = 234	234 ≥ 25: Sim; 25×1=25, 25×2=50, 25×3=75, 25×4=100, 25×5=125, 25×6=150, 25×7=175, 25×8=200, 25×9=225	234 − 225 = 9	295×10+9 = 2959
6: 9×10+5 = 95	95 ≥ 25: Sim; 25×1=25, 25×2=50, 25×3=75, 25×4=100 > 95 → usar 3	95 − 75 = 20	2959×10+3 = 29593

Passo e número	Verificar e tabuada	Resto	Quociente
7: 20×10+1 = 201	201 ≥ 25: Sim; 25×1=25, 25×2=50, 25×3=75, 25×4=100, 25×5=125, 25×6=150, 25×7=175, 25×8=200, 25×9=225 > 201 → usar 8	201 − 200 = 1	29593×10+8 = 295938
8: 1×10+0 = 10	10 ≥ 25: Não; algarismo 0	10	295938×10+0 = 2959380
9: 10×10+7 = 107	107 ≥ 25: Sim; 25×1=25, 25×2=50, 25×3=75, 25×4=100, 25×5=125 > 107 → usar 4	107 − 100 = 7	2959380×10+4 = 29593804
10: 7×10+8 = 78	78 ≥ 25: Sim; 25×1=25, 25×2=50, 25×3=75, 25×4=100 > 78 → usar 3	78 − 75 = 3	29593804×10+3 = 295938043

Resultado: quociente 295938043 e resto 3 (25 × 295938043 + 3 = 7398451078).

Números negativos e reais

Para dividir números reais usa-se a mesma divisão inteira. Primeiro igualam-se as casas decimais do dividendo e do divisor, acrescentando zeros ao que tiver menos casas (cada zero é uma multiplicação por 10 do array). Assim, 125,5 ÷ 2,5 passa a 1255 ÷ 25 e 7,5 ÷ 2 passa a 75 ÷ 20. O quociente inteiro não muda; o resto fica com o mesmo número de casas decimais que foram igualadas (75 ÷ 20 dá resto 15, ou seja 1,5).

O sinal trata-se à parte, com os valores absolutos: o quociente é negativo quando os sinais do dividendo e do divisor são diferentes, e o resto tem o sinal do dividendo. Assim verifica-se sempre dividendo = divisor × quociente + resto (por exemplo, −125,5 = 2,5 × (−50) + (−0,5)). O divisor não pode ser 0.

Raiz quadrada

A raiz quadrada também se calcula algarismo a algarismo:

- Agrupam-se os algarismos 2 a 2 a partir da vírgula: para a esquerda na parte inteira e para a direita na parte decimal. Se faltar um algarismo num grupo, acrescenta-se um 0.
- Para cada grupo: resto = resto × 100 + grupo.

– Procura-se o maior algarismo a tal que (2 × R × 10 + a) × a ≤ resto, onde R é a raiz já calculada (na aula, "2a × a").

– Resto = resto − (2 × R × 10 + a) × a e R = R × 10 + a.

– A raiz tem uma casa decimal por cada grupo da parte decimal.

Exemplo da aula: raiz de 341,7. Os grupos são 03 41 , 70.

Grupo	Resto	Tentativas (2R×10 + a) × a	Novo resto e R
1: 03	3	1×1=1; 2×2=4 > 3 → usar 1	3 − 1 = 2; R = 1
2: 41	2×100+41 = 241	27×7=189; 28×8=224; 29×9=261 > 241 → usar 8	241 − 224 = 17; R = 18
3: 70	17×100+70 = 1770	363×3=1089; 364×4=1456; 365×5=1825 > 1770 → usar 4	1770 − 1456 = 314; R = 184

Resultado: raiz 18,4 e resto 3,14 ($18{,}4^2 + 3{,}14 = 338{,}56 + 3{,}14 = 341{,}70$). Para obter mais casas decimais, basta escrever mais zeros na parte decimal (por exemplo, 2,0000 dá 1,41).

Representação dos números em arrays

Cada número fica num array de 26 bytes, alinhado à direita: a última posição tem as unidades e as posições da esquerda ficam a 0. Com esta forma, as operações sobre os arrays são simples:

– empurra: X = X × 10 + d (cada algarismo anda uma posição para a esquerda e d entra nas unidades).

– compara: compara dois arrays do algarismo mais significativo para o menos; a primeira diferença decide.

– subtrai: X = X − Y, da direita para a esquerda, com empréstimo.

– mul_dig: Y = X × k, com k de 0 a 9, da direita para a esquerda, com transporte (as dezenas contam-se subtraindo 10).

– escreve_arr: escreve o array sem os zeros à esquerda e com a vírgula antes das casas decimais.

PSEUDOCÓDIGO

O pseudocódigo está escrito na linguagem GAL (Generic Algorithm Language) do Algorithmi. Os arrays definem-se com o tamanho entre parênteses retos e lerCaracter() representa a leitura de uma tecla (INT 21h, função 01h). As variáveis neg e ndec, preenchidas por ler_num, são globais.

Programa da divisão inteira

```
inicio Divisão
    definir inteiro A[26]              // dividendo: um algarismo por posição
    definir inteiro B[26]              // divisor
    definir inteiro P[26]              // número de cada passo (e resto)
    definir inteiro PR[26]             // divisor x k (tabuada)
    definir inteiro OK[26]             // maior produto que não passa P
    definir inteiro Q[26]              // quociente
    definir inteiro buf[12]            // algarismos escritos
    definir inteiro nd = 0
    definir inteiro sa = 0             // sinal do dividendo (1 = negativo)
    definir inteiro sb = 0             // sinal do divisor
    definir inteiro da = 0             // casas decimais do dividendo
    definir inteiro db = 0             // casas decimais do divisor
    definir inteiro dec = 0
    definir inteiro i = 0
    definir inteiro k = 0
    definir inteiro fim = 0
    faz
        escrever "Dividendo: "
        executar nd = ler_num(buf)
    enquanto nd == 0
    executar sa = neg
    executar da = ndec
    executar para_array(A, buf, nd)
    faz
        escrever "Divisor: "
        executar nd = ler_num(buf)
        se nd > 0
            executar para_array(B, buf, nd)
            se e_zero(B) == 1
                escrever "Erro: o divisor não pode ser 0"
                executar nd = 0
            fim se
        fim se
    enquanto nd == 0
    executar sb = neg
    executar db = ndec
    enquanto da < db faz               // igualar as casas decimais
        executar empurra(A, 0)         // A = A x 10
        executar da = da + 1
    fim enquanto
    enquanto db < da faz
        executar empurra(B, 0)         // B = B x 10
        executar db = db + 1
    fim enquanto
    executar dec = da
    enquanto i < 25 e A[i] == 0 faz    // 1.º algarismo diferente de 0
        executar i = i + 1
    fim enquanto
    enquanto i < 26 faz                // um passo por algarismo
        executar empurra(P, A[i])      // P = P x 10 + A[i]
```

```
        escrever "número = " + P
        executar k = 0
        se compara(P, B) >= 0
            executar fim = 0
            enquanto fim == 0 faz      // tabuada do divisor
                executar k = k + 1
                executar mul_dig(B, k, PR)
                escrever B + " x " + k + " = " + PR
                se compara(PR, P) > 0
                    executar k = k - 1 // passou: usa o anterior
                    executar fim = 1
                senão
                    executar copia(PR, OK)
                    se k == 9
                        executar fim = 1
                    fim se
                fim se
            fim enquanto
            executar subtrai(P, OK)    // resto = P - divisor x k
            escrever "resto = " + P
        fim se
        executar empurra(Q, k)         // Q = Q x 10 + k
        escrever "quociente = " + Q
        executar i = i + 1
    fim enquanto
    escrever "Quociente inteiro = "
    se sa != sb e e_zero(Q) == 0
        escrever "-"
    fim se
    executar escreve_arr(Q, 0)
    escrever "Resto = "
    se sa == 1 e e_zero(P) == 0
        escrever "-"
    fim se
    executar escreve_arr(P, dec)       // o resto tem dec casas decimais
fim Divisão
```

Programa da raiz quadrada

```
inicio Raiz quadrada
    definir inteiro buf[12]            // algarismos escritos
    definir inteiro G[26]              // algarismos agrupados 2 a 2
    definir inteiro R[26]              // raiz já calculada
    definir inteiro RS[26]             // resto
    definir inteiro DB[26]             // 2 x R
    definir inteiro T[26]              // 2 x R x 10 + a
    definir inteiro PR[26]             // (2 x R x 10 + a) x a
    definir inteiro OK[26]             // maior produto que não passa RS
    definir inteiro nd = 0
    definir inteiro ni = 0             // algarismos da parte inteira
    definir inteiro ng = 0             // algarismos em G
    definir inteiro dr = 0             // casas decimais da raiz
    definir inteiro i = 0
    definir inteiro a = 0
    definir inteiro m = 0              // maior a que serve
    definir inteiro fim = 0
    faz
        escrever "Número: "
        executar nd = ler_num(buf)
        se neg == 1
            escrever "Um número negativo não tem raiz real"
            executar nd = 0
        fim se
    enquanto nd == 0
    executar ni = nd - ndec            // agrupar 2 a 2 a partir da vírgula
    se ni % 2 == 1
        executar G[ng] = 0             // 0 à esquerda
        executar ng = ng + 1
    fim se
    enquanto i < nd faz
        executar G[ng] = buf[i]
        executar ng = ng + 1
        executar i = i + 1
    fim enquanto
    se ndec % 2 == 1
        executar G[ng] = 0             // 0 à direita
        executar ng = ng + 1
        executar ndec = ndec + 1
    fim se
    executar dr = ndec / 2             // um algarismo decimal por grupo
    executar i = 0
    enquanto i < ng faz                // um passo por grupo
        executar empurra(RS, G[i])
        executar empurra(RS, G[i + 1]) // RS = RS x 100 + grupo
        escrever "resto = " + RS
        executar mul_dig(R, 2, DB)     // DB = 2 x R
        executar a = 0
        executar m = 0
        executar zera(OK)
        executar fim = 0
```

```
        enquanto fim == 0 faz
            executar a = a + 1
            executar copia(DB, T)
            executar empurra(T, a)     // T = 2 x R x 10 + a
            executar mul_dig(T, a, PR) // PR = T x a
            escrever T + " x " + a + " = " + PR
            se compara(PR, RS) > 0
                executar fim = 1       // passou: usa o anterior
            senão
                executar copia(PR, OK)
                executar m = a
                se a == 9
                    executar fim = 1
                fim se
            fim se
        fim enquanto
        executar subtrai(RS, OK)       // RS = RS - OK
        escrever "resto = " + RS
        executar empurra(R, m)         // R = R x 10 + m
        escrever "R = " + R
        executar i = i + 2
    fim enquanto
    escrever "Raiz quadrada = "
    executar escreve_arr(R, dr)
    escrever "Resto = "
    executar escreve_arr(RS, 2 * dr)
fim Raiz quadrada
```

Rotina ler_num

```
função inteiro ler_num(inteiro buf[12])
    // devolve o número de algarismos (0 = inválido);
    // as variáveis globais neg e ndec ficam com o sinal e as casas decimais
    definir inteiro nd = 0
    definir inteiro virg = 0          // 1 depois da vírgula
    definir inteiro mau = 0           // 1 se o número for inválido
    definir inteiro c = 0
    definir inteiro d = 0
    executar neg = 0
    executar ndec = 0
    executar c = lerCaracter()
    se c == 45                        // '-'
        executar neg = 1
        executar c = lerCaracter()
    fim se
    enquanto c != 13 faz              // até Enter
        se c == 44 ou c == 46         // ',' ou '.'
            se virg == 1
                executar mau = 1      // segunda vírgula
            fim se
            executar virg = 1
        senão
            executar d = c - 48
            se d >= 0 e d <= 9
                se nd < 12
                    executar buf[nd] = d
                    executar nd = nd + 1
                    se virg == 1
                        executar ndec = ndec + 1
                    fim se
                senão
                    executar mau = 1   // algarismos a mais
                fim se
            fim se
        fim se
        executar c = lerCaracter()
    fim enquanto
    se mau == 1
        retornar 0
    fim se
    retornar nd
fim ler_num
```

Rotina para_array

```
função vazio para_array(inteiro X[26], inteiro buf[12], inteiro nd)
    definir inteiro i = 0
    executar zera(X)
    enquanto i < nd faz
        executar empurra(X, buf[i])    // X = X x 10 + algarismo
        executar i = i + 1
    fim enquanto
fim para_array
```

Rotina empurra

```
função vazio empurra(inteiro X[26], inteiro d)
    // X = X x 10 + d
    definir inteiro i = 0
    enquanto i < 25 faz
        executar X[i] = X[i + 1]       // cada algarismo anda uma posição
        executar i = i + 1
    fim enquanto
    executar X[25] = d
fim empurra
```

Rotina compara

```
função inteiro compara(inteiro X[26], inteiro Y[26])
    // 1 se X > Y, 0 se X = Y, -1 se X < Y
    definir inteiro i = 0
    enquanto i < 26 faz
        se X[i] > Y[i]
            retornar 1
        fim se
        se X[i] < Y[i]
            retornar -1
        fim se
        executar i = i + 1
    fim enquanto
    retornar 0
fim compara
```

Rotina subtrai

```
função vazio subtrai(inteiro X[26], inteiro Y[26])
    // X = X - Y  (X >= Y)
    definir inteiro i = 25
    definir inteiro e = 0              // empréstimo
    definir inteiro d = 0
    enquanto i >= 0 faz
        executar d = X[i] - e - Y[i]
        executar e = 0
        se d < 0
            executar d = d + 10        // pede 10 emprestado
            executar e = 1
        fim se
        executar X[i] = d
        executar i = i - 1
    fim enquanto
fim subtrai
```

Rotina mul_dig

```
função vazio mul_dig(inteiro X[26], inteiro k, inteiro Y[26])
    // Y = X x k  (k de 0 a 9)
    definir inteiro i = 25
    definir inteiro t = 0              // transporte
    definir inteiro p = 0
    enquanto i >= 0 faz
        executar p = X[i] * k + t
        executar t = 0
        enquanto p >= 10 faz           // conta as dezenas sem dividir
            executar p = p - 10
            executar t = t + 1
        fim enquanto
        executar Y[i] = p
        executar i = i - 1
    fim enquanto
fim mul_dig
```

Rotina escreve_arr

```
função vazio escreve_arr(inteiro X[26], inteiro nd)
    // escreve X sem zeros à esquerda, com nd casas decimais
    definir inteiro u = 25 - nd        // posição das unidades
    definir inteiro i = 0
    enquanto i < u e X[i] == 0 faz     // salta os zeros à esquerda
        executar i = i + 1
    fim enquanto
    enquanto i < 26 faz
        escrever X[i]
        se i == u e nd > 0
            escrever ","
        fim se
        executar i = i + 1
    fim enquanto
fim escreve_arr
```

Rotinas zera, copia e e_zero

```
função vazio zera(inteiro X[26])
    definir inteiro i = 0
    enquanto i < 26 faz
        executar X[i] = 0
        executar i = i + 1
    fim enquanto
fim zera

função vazio copia(inteiro X[26], inteiro Y[26])
    definir inteiro i = 0              // Y = X
    enquanto i < 26 faz
        executar Y[i] = X[i]
        executar i = i + 1
    fim enquanto
fim copia

função inteiro e_zero(inteiro X[26])
    definir inteiro i = 0              // 1 se todos forem 0
    enquanto i < 26 faz
        se X[i] != 0
            retornar 0
        fim se
        executar i = i + 1
    fim enquanto
    retornar 1
fim e_zero
```

FLUXOGRAMAS

Os fluxogramas seguem a notação do Algorithmi: terminadores para o início e o fim, a forma com barras duplas para definir variáveis, retângulos para executar, a forma de cartão para escrever e losangos para as decisões. Os programas principais estão divididos em várias figuras, ligadas por conectores numerados (1, 2, 3, …); na rotina ler_num os conectores são A e B.

Figura 1: Divisão (1/6): ler o dividendo

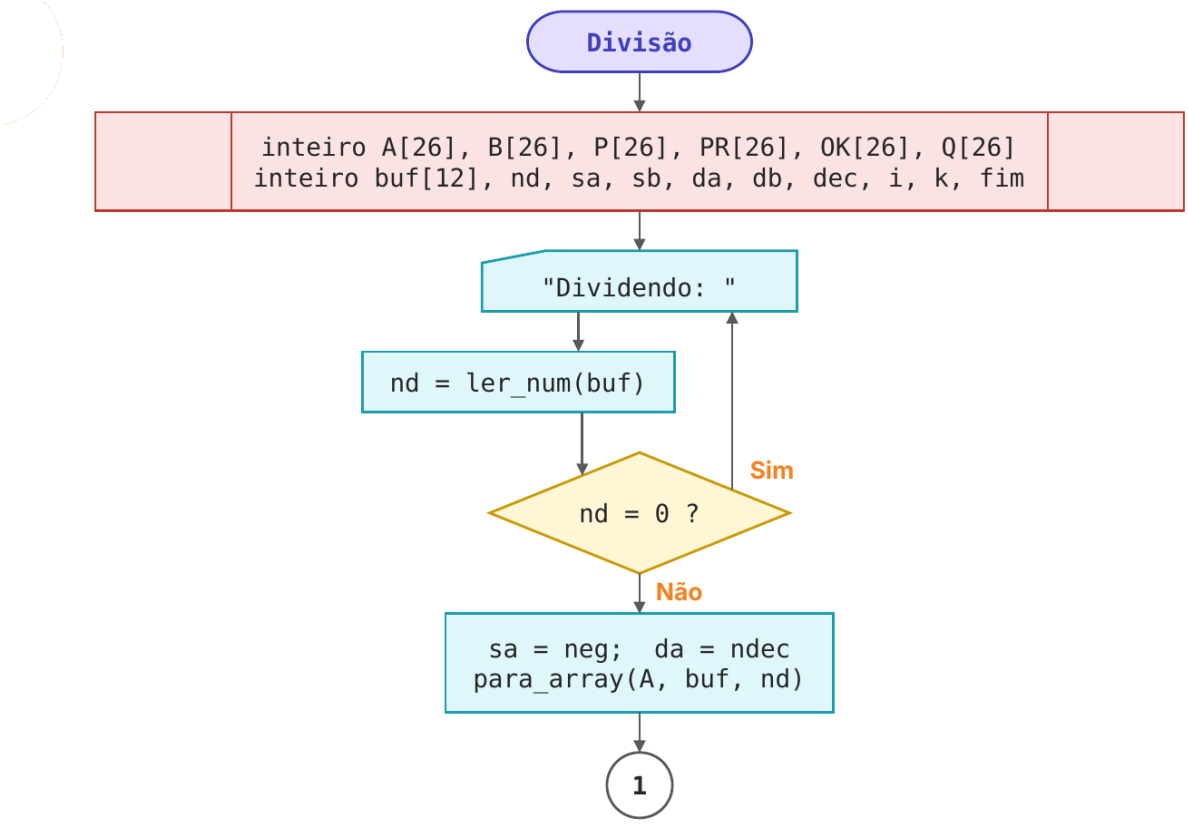


Figura 2: Divisão (2/6): ler o divisor

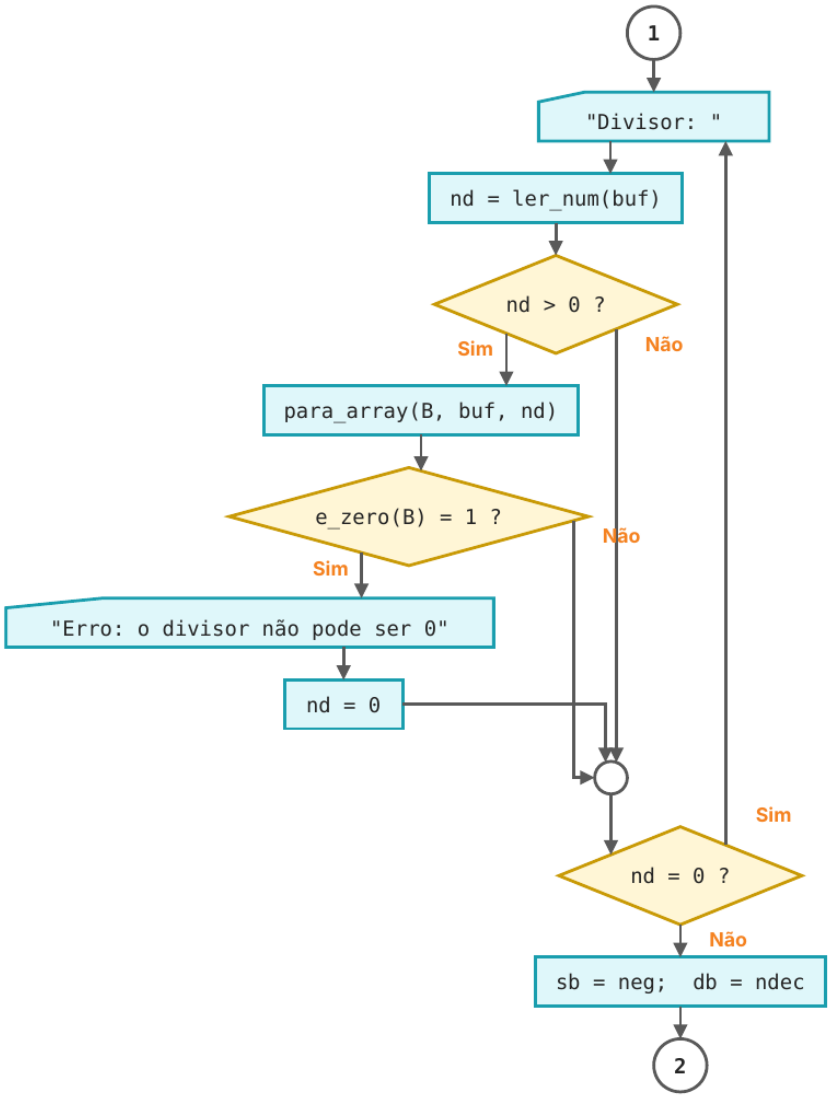


Figura 3: Divisão (3/6): igualar as casas decimais

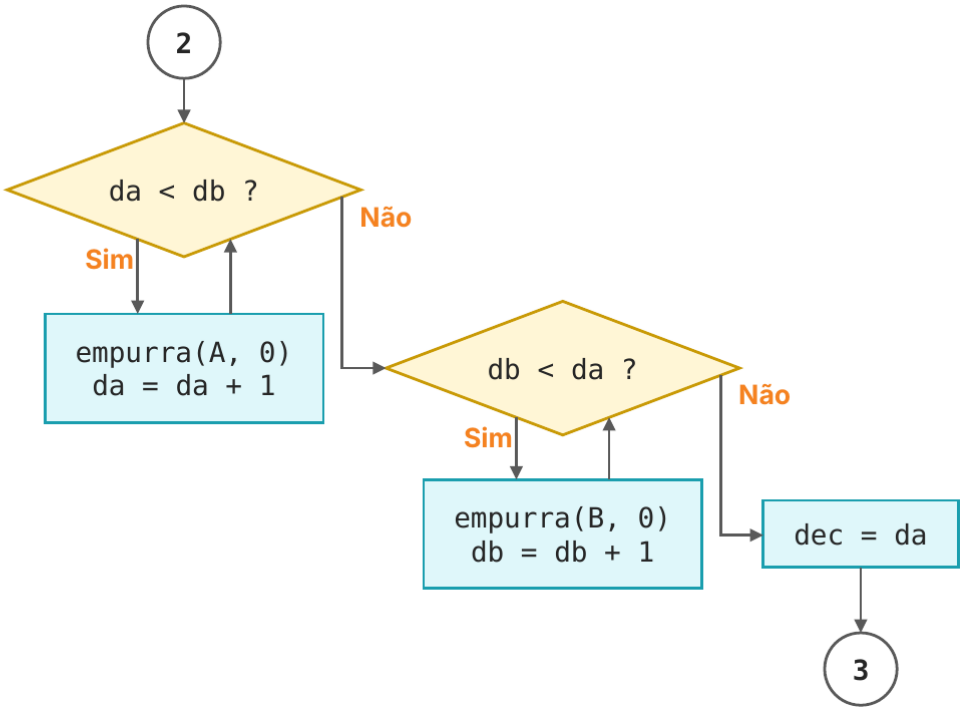


Figura 4: Divisão (4/6): um passo por algarismo

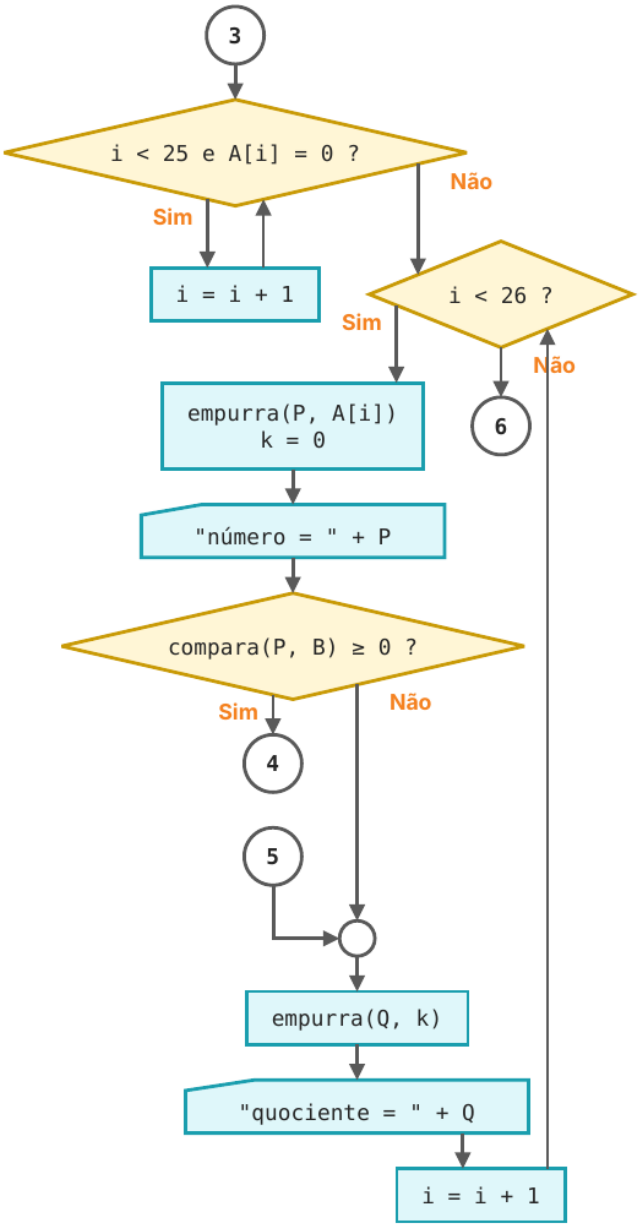


Figura 5: Divisão (5/6): tabuada do divisor e resto

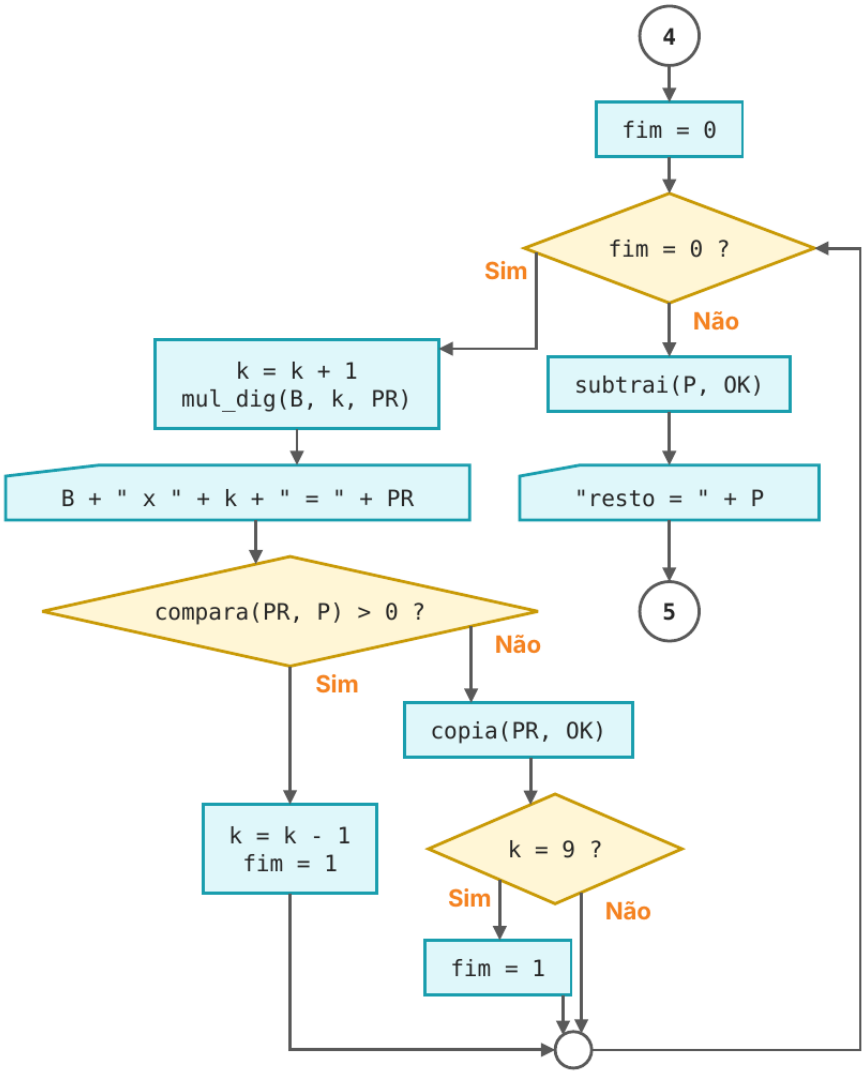


Figura 6: Divisão (6/6): resultado final

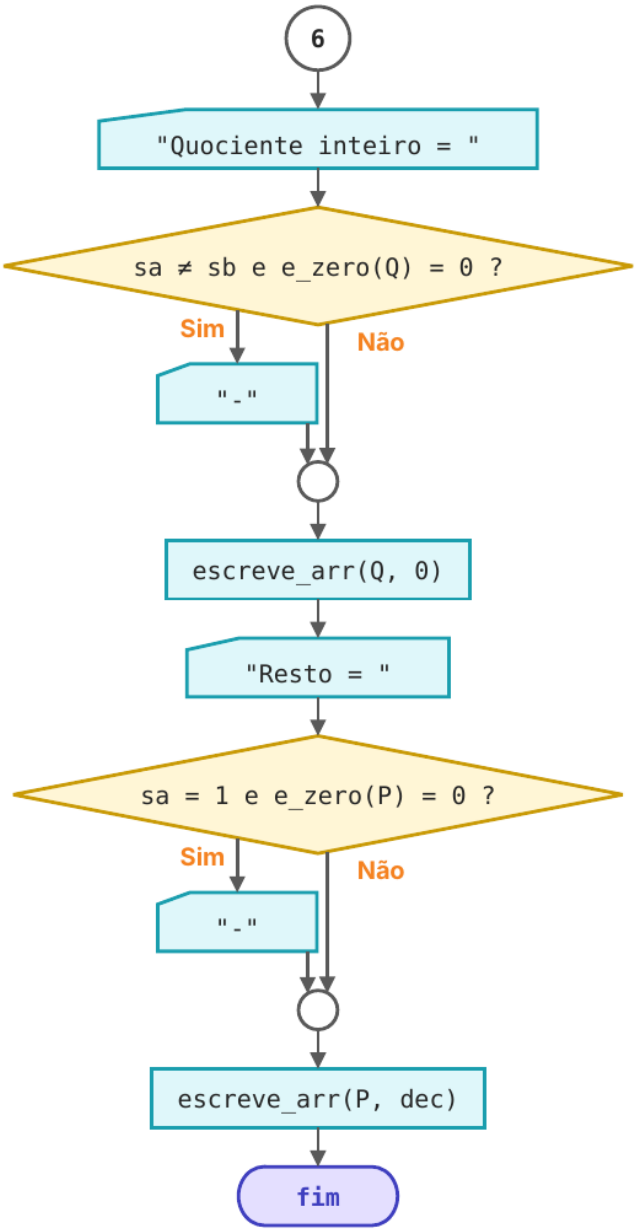


Figura 7: Raiz quadrada (1/4): ler o número

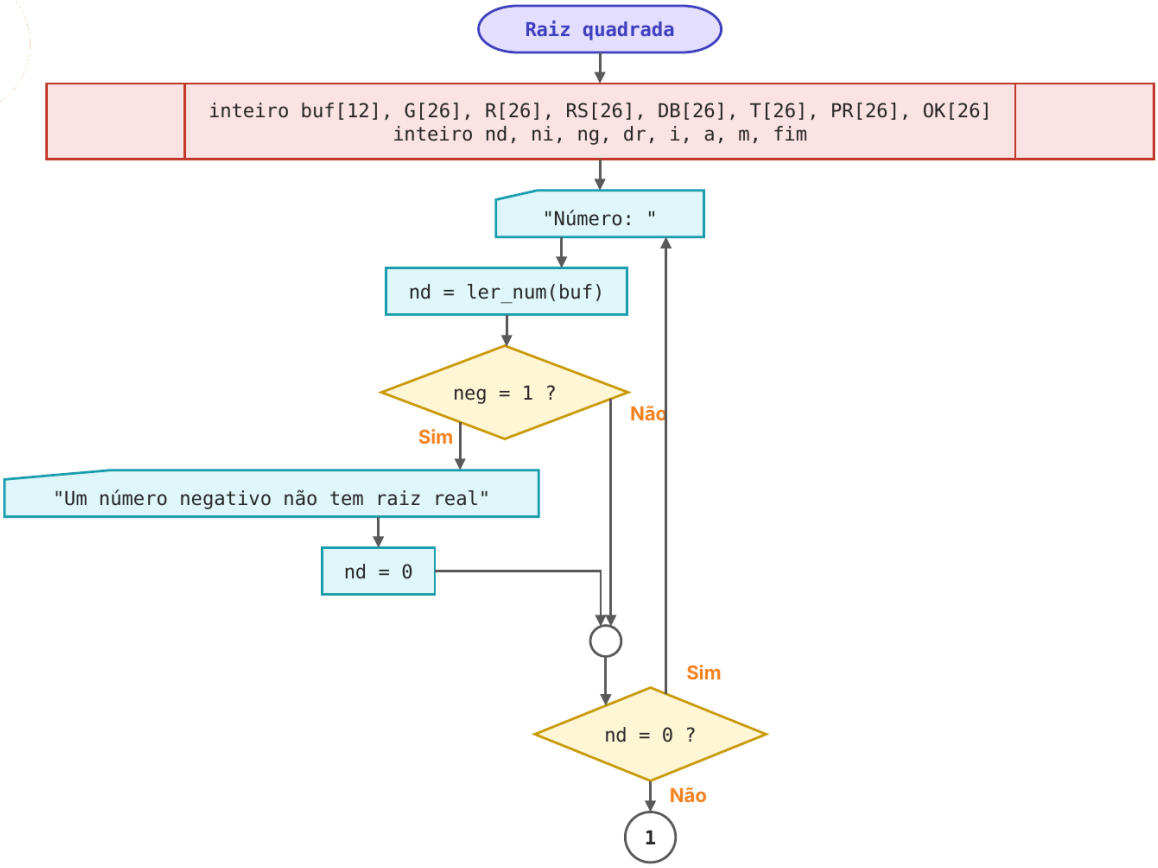


Figura 8: Raiz quadrada (2/4): agrupar os algarismos 2 a 2

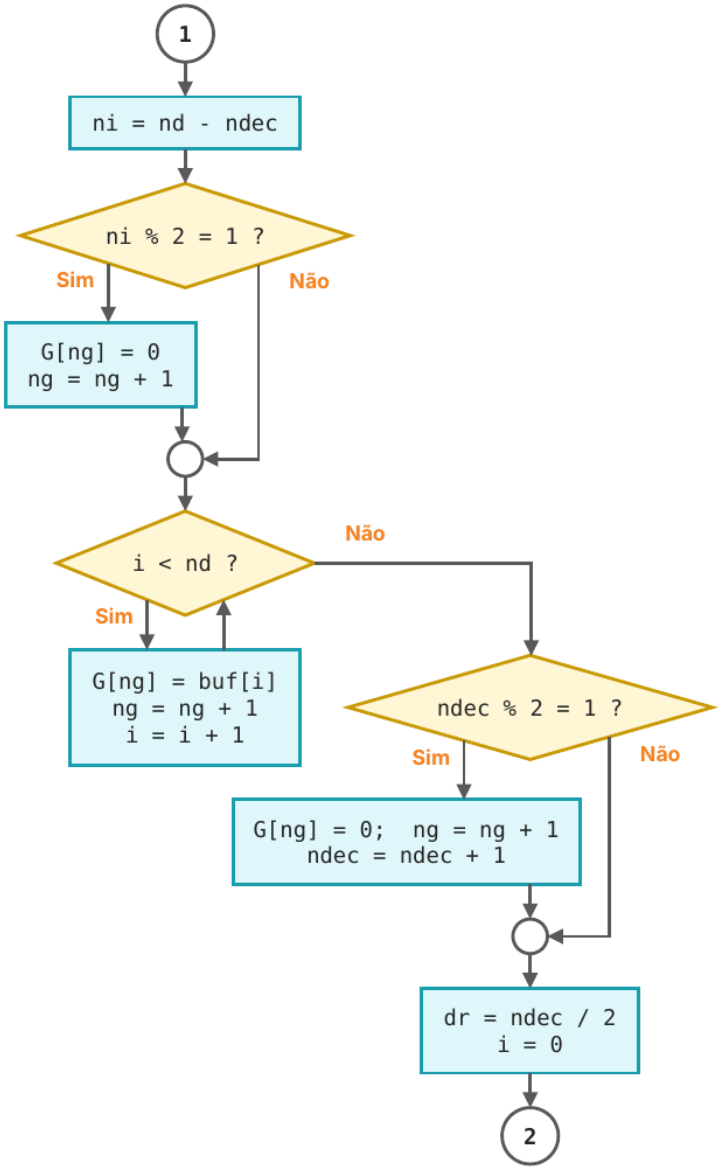


Figura 9: Raiz quadrada (3/4): um passo por grupo

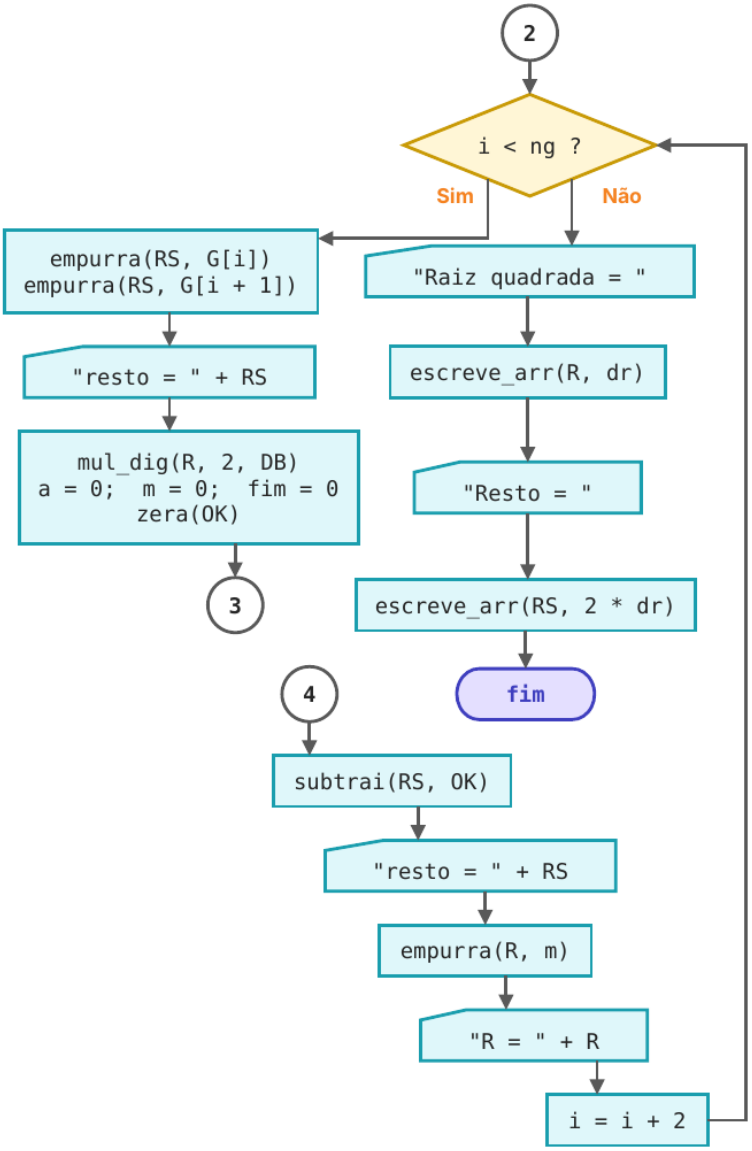


Figura 10: Raiz quadrada (4/4): procurar o algarismo a

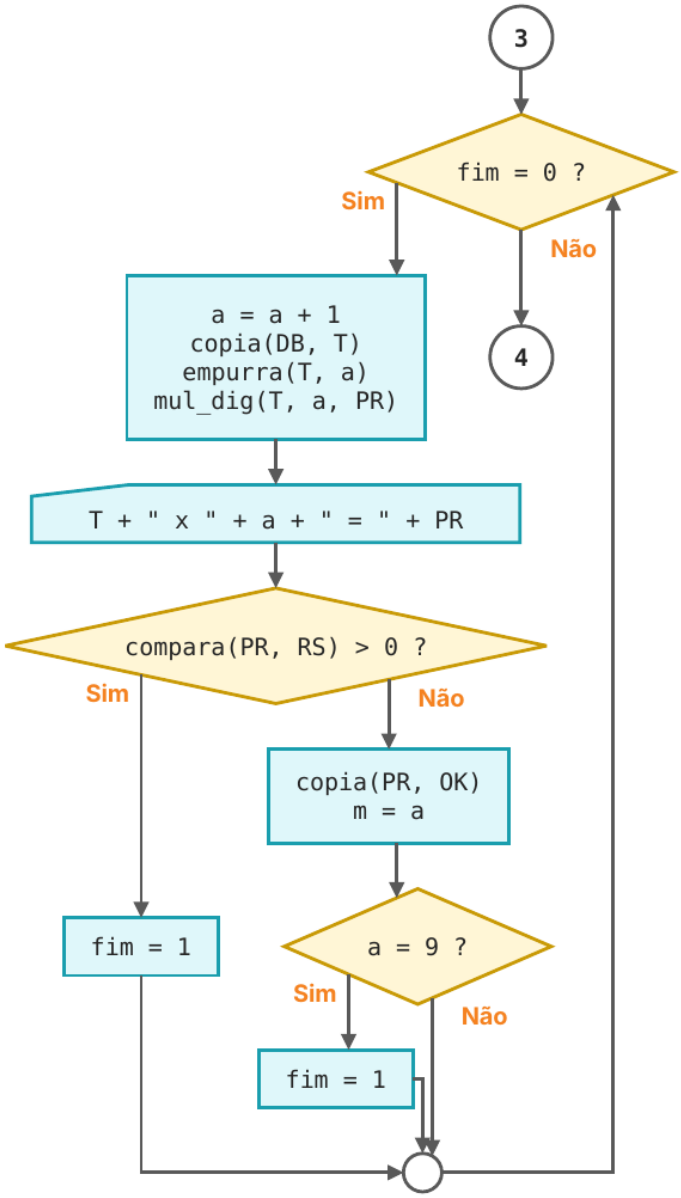


Figura 11: Rotina ler_num (1/2): ler um número para o array

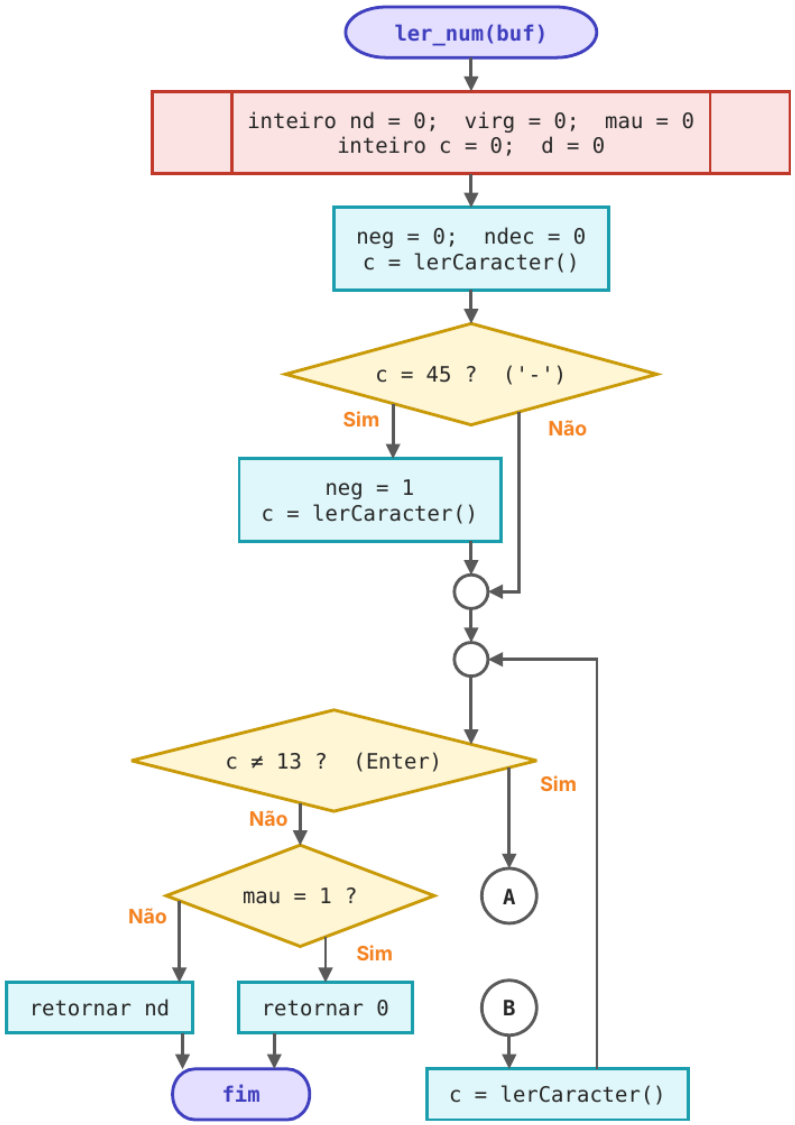


Figura 12: Rotina ler_num (2/2): tratar cada carácter

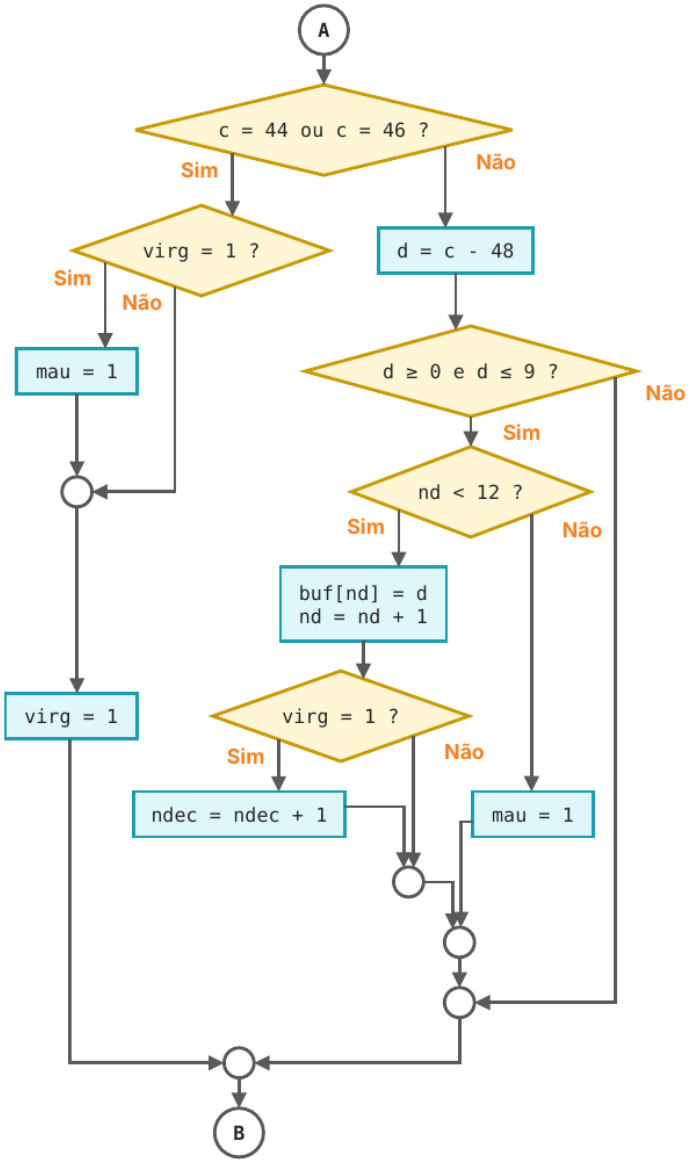
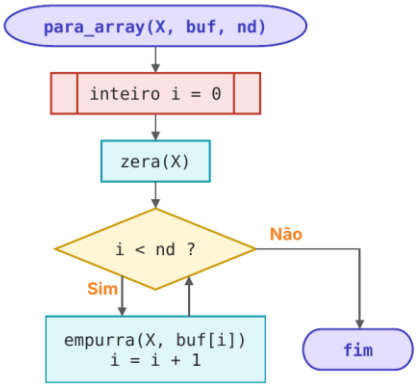


Figura 13: Rotinas para_array e empurra



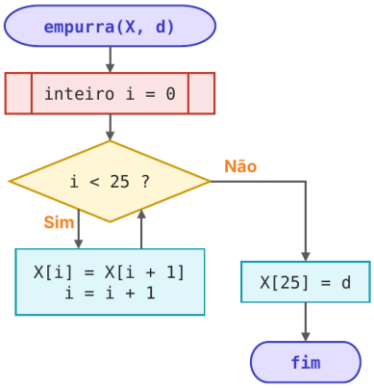


Figura 14: Rotina compara: comparar dois arrays

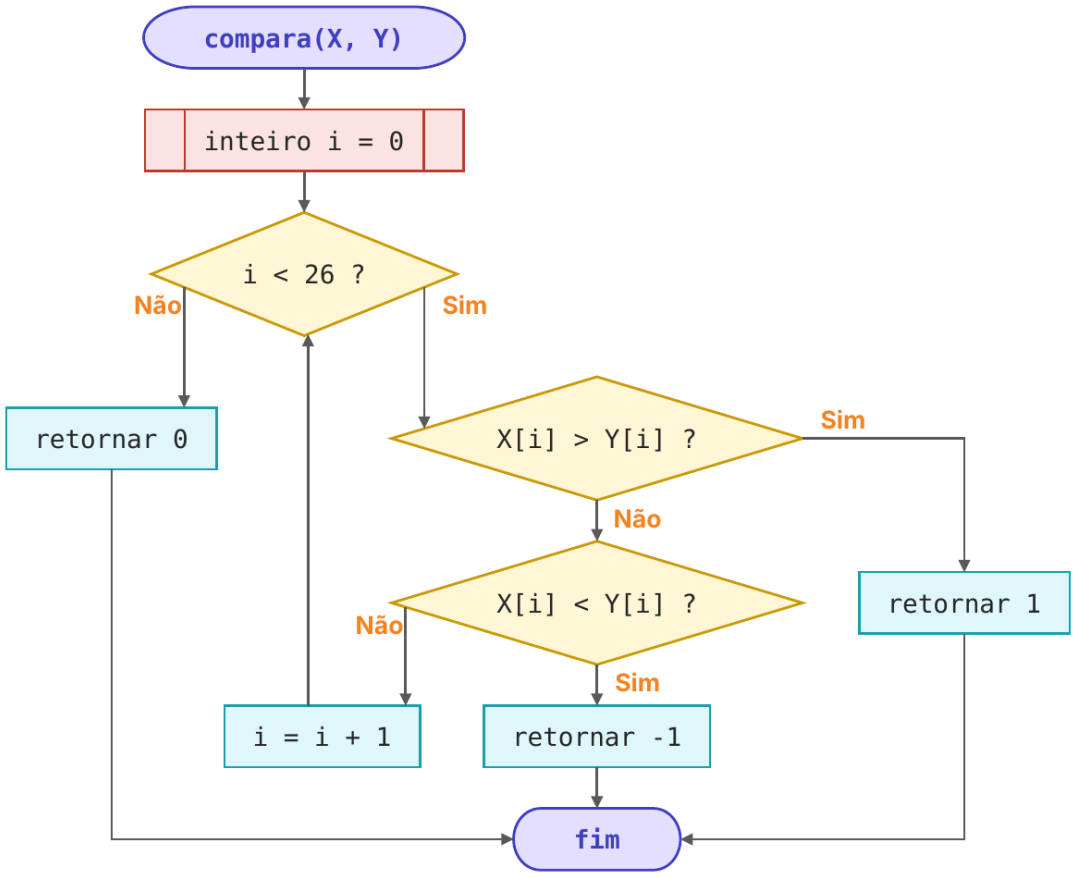


Figura 15: Rotina subtrai: X = X − Y

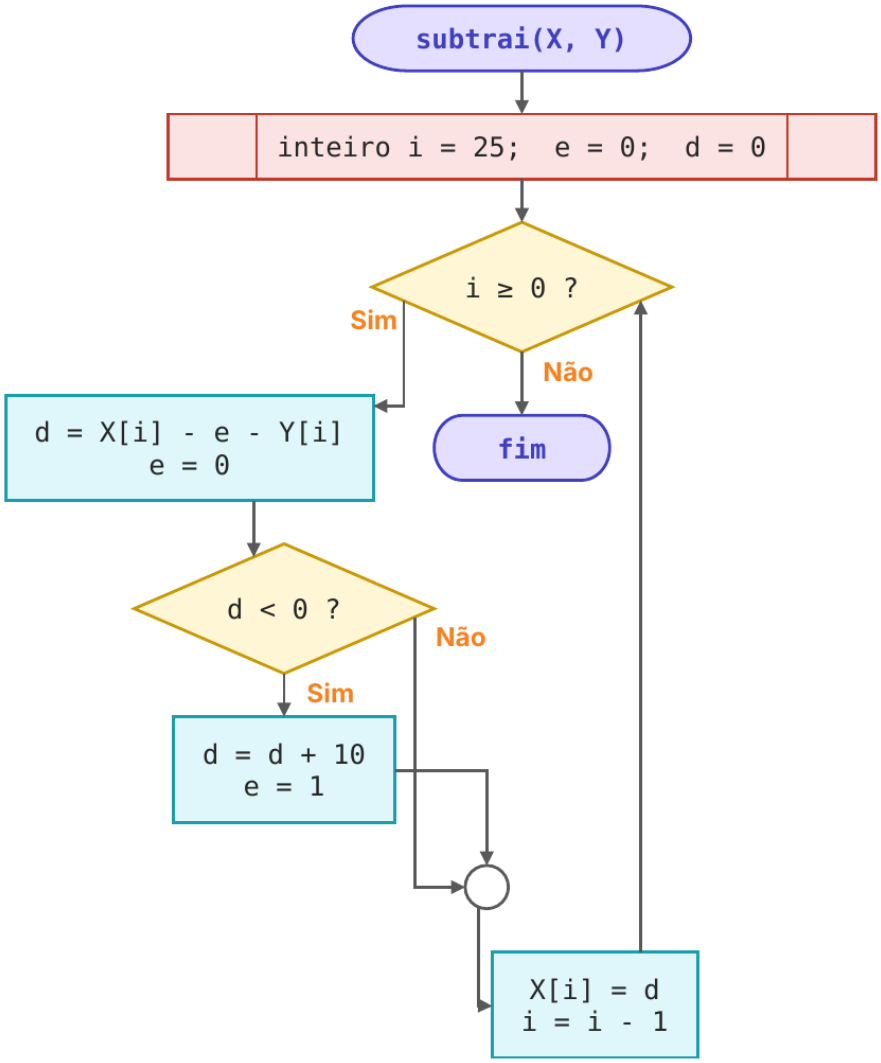


Figura 16: Rotina mul_dig: Y = X × k (k de 0 a 9)

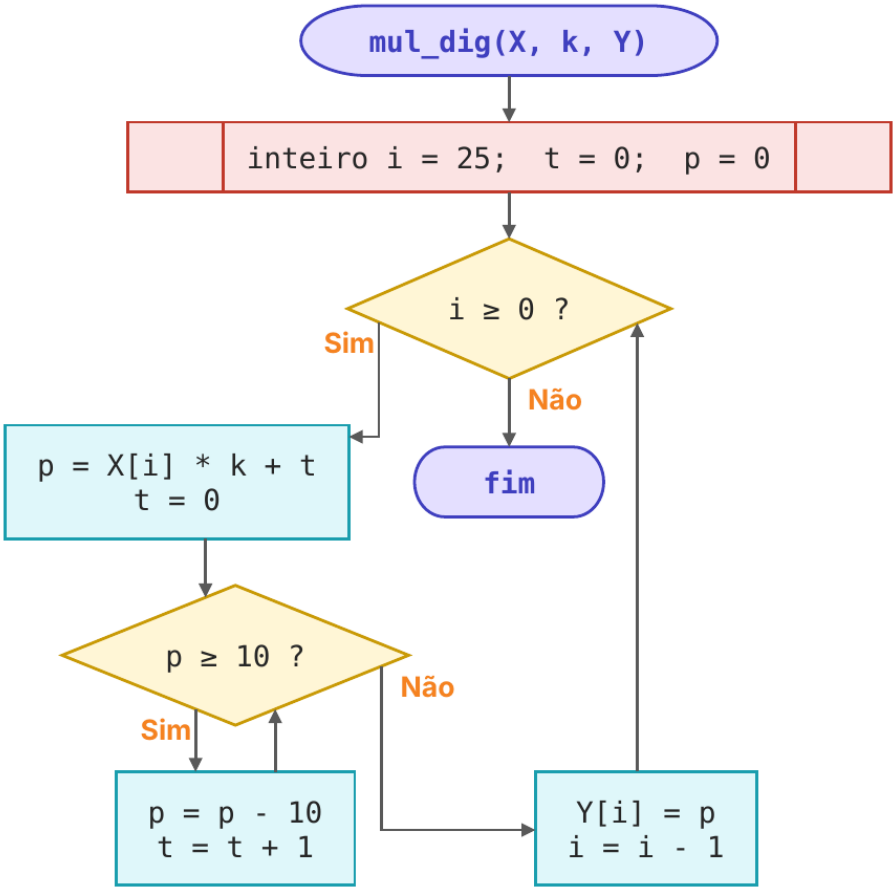


Figura 17: Rotina escreve_arr: escrever um array

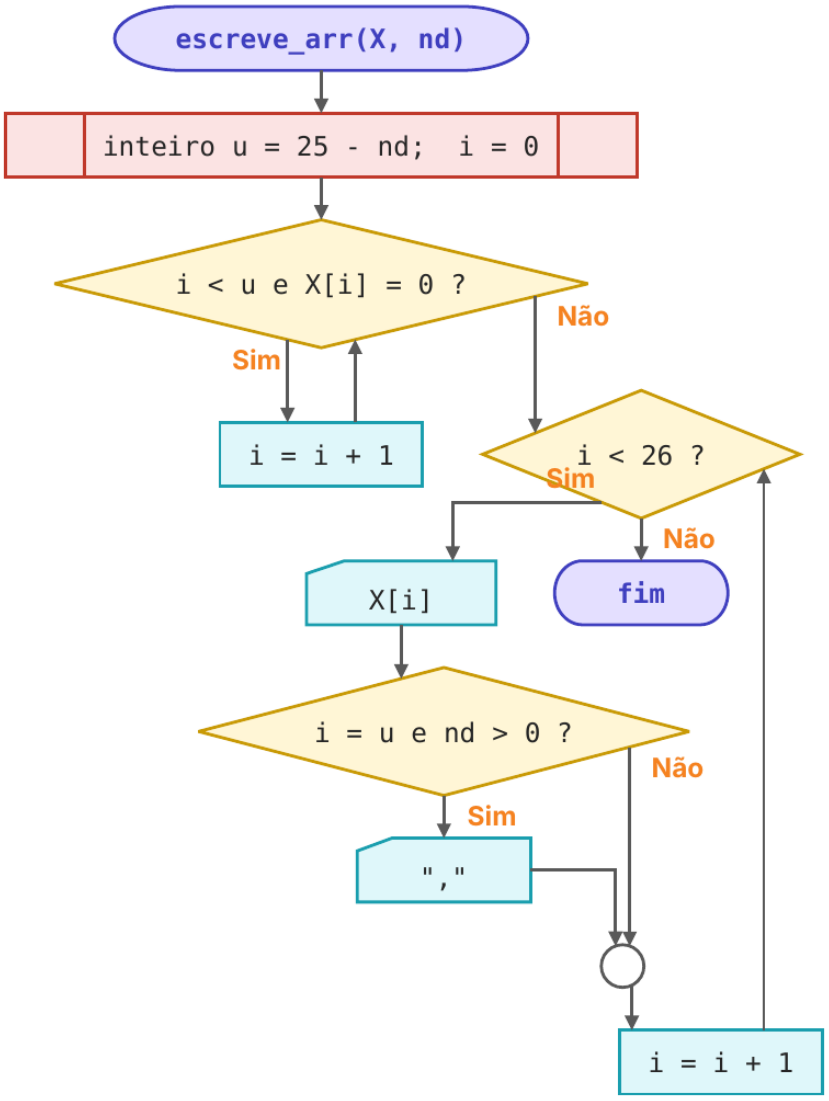
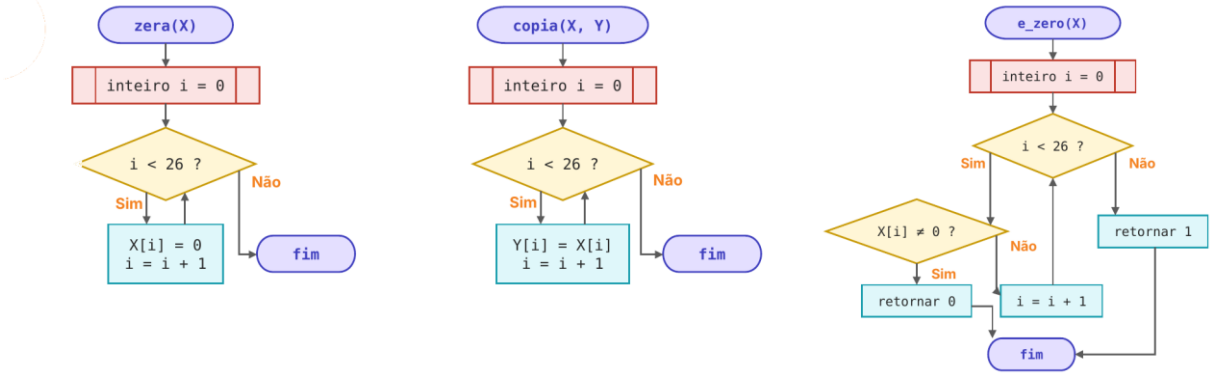


Figura 18: Rotinas zera, copia e e_zero



INSTRUÇÕES UTILIZADAS

As descrições baseiam-se no capítulo 6 («The 80x86 Instruction Set») do livro The Art of Assembly Language Programming, de Randall Hyde. A tabela indica onde cada instrução é usada nos dois programas.

Registos usados

Registo	Utilização
AX (AH/AL)	Algarismo lido ou a escrever; produto de mul (AX = AL × DL); AH = função do DOS; AH = empréstimo em subtrai.
BX	Endereço de um array (lea bx, ...) ou índice da posição (de 0 a LEN−1).
CX	Contador de ciclos (loop) e número de casas decimais em escreve_arr.
DX	Endereço do texto a escrever (função 09h); DL = carácter (função 02h); DL = multiplicador e DH = transporte em mul_dig.
SI, DI	Endereços dos arrays de origem e de destino (copia, compara, subtrai, mul_dig) e índices.
DS	Segmento de dados, carregado com @data no início.
Flags (CF, ZF, SF)	CF = 1 marca número inválido em ler_num; compara deixa as flags como cmp; SF indica empréstimo em subtrai.

Transferência de dados e pilha

Instrução	O que faz	Onde é usada	Secção
MOV dest, orig	Copia orig para dest.	Em todo o código: mov al, [si+bx]; mov [di+bx], al.	6.3.1
LEA reg, mem	Carrega o endereço (offset) de mem.	lea bx, NUMA; lea dx, msgA.	6.3.4
PUSH / POP	Guarda e recupera um registo na pilha.	No início e no fim de cada rotina.	6.3.5

Aritméticas

Instrução	O que faz	Onde é usada	Secção
ADD dest, orig	dest = dest + orig.	add al, 10 (empréstimo); add al, dh (transporte); add dl, '0'.	6.5.1
SUB dest, orig	dest = dest − orig.	sub al, [di+bx] (subtrai); sub al, 10 (dezenas); sub al, '0'.	6.5.2
MUL orig	AX = AL × orig (sem sinal).	mul dl em mul_dig: algarismo × algarismo (no máximo 81).	6.5.6
INC / DEC	Soma ou subtrai 1.	Índices, contadores e o transporte (inc dh).	6.5.1 e 6.5.2
CMP a, b	Calcula a − b só para as flags.	cmp al, 13 (Enter); cmp al, [di] (compara); cmp al, 10.	6.5.3

Lógicas

Instrução	O que faz	Onde é usada	Secção
XOR reg, reg	Põe o registo a 0.	xor si, si; xor ah, ah.	6.6.1
OR al, mem	OU lógico; afeta ZF.	e_zero: junta todos os algarismos para saber se são 0.	6.6.1
TEST al, 1	E lógico só para as flags.	Saber se o número de algarismos é ímpar (raiz).	6.6.4.1

Controlo de fluxo e flags

Instrução	O que faz	Onde é usada	Secção
JMP	Salto incondicional.	Ciclos (jmp ciclo, jmp c_tab).	6.9.1
JE / JNE	Salta se igual / diferente (ZF).	Enter, vírgula, sinal, comparação de algarismos.	6.9.4
JA / JAE / JBE	Saltos sem sinal (CF e ZF).	Depois de compara: o produto passou o número? (ja); algarismo < 10 (jbe).	6.9.4
JNS	Salta se SF = 0.	subtrai e mul_dig: repetir enquanto o índice ≥ 0.	6.9.4
JNC	Salta se CF = 0.	Depois de ler_num: o número é válido?	6.9.4
LOOP	CX = CX − 1 e salta se CX ≠ 0.	Percorrer os arrays (zera, copia, empurra, compara).	6.9.6
CALL / RET	Chama uma rotina e volta.	Todas as rotinas.	6.9.2
INT 21h	Chama uma função do DOS (AH = número da função).	Ler teclas, escrever e terminar.	6.9.3
CLC / STC	Põe CF a 0 / a 1.	ler_num: número válido (clc) ou inválido (stc).	6.10

Funções do DOS (INT 21h)

Função	O que faz	Onde é usada	Secção
AH = 01h	Lê uma tecla com eco; devolve o código em AL.	ler_num.	13.3.2
AH = 02h	Escreve o carácter que está em DL.	escreve_dig, escreve_arr, menos.	13.3.2
AH = 09h	Escreve o texto terminado em '$' cujo endereço está em DX.	Rotina texto (mensagens).	13.3.2
AH = 4Ch	Termina o programa.	Fim do programa principal.	13.3.7.1

Diretivas e declarações

Diretiva	O que faz	Onde é usada	Secção
.model small, .stack 100h	Modelo de memória e tamanho da pilha.	Início dos dois programas.	8.8.7
EQU	Define uma constante.	LEN equ 26; MAXD equ 12.	cap. 8
DB / DW, DUP	Declara bytes / palavras; DUP repete um valor.	Arrays: NUMA db LEN dup(0); textos.	5.3.1, 5.6.1.1
PROC / ENDP, END	Início e fim de uma rotina; fim do programa.	Todas as rotinas; end main.	8.7 e 8.9

Nota: as instruções são praticamente as mesmas do trabalho anterior (Tarefa Laboratorial #04). A instrução DIV não é usada em nenhum dos programas, como pede o enunciado, e MUL só multiplica um algarismo por outro algarismo (o resultado cabe num byte).

Nota: os saltos condicionais do 8086 só alcançam −128 a +127 bytes. Nos dois ciclos principais, que são maiores, usa-se a condição contrária seguida de JMP (por exemplo: ja fim_ciclo / jmp ciclo).

TESTES E RESULTADOS

Os valores esperados foram calculados à parte. A última coluna mostra o que o programa escreveu ao ser executado.

Divisão inteira

Teste	Entrada	Resultado esperado	Resultado obtido
1	7398451078 ÷ 25	quociente 295938043; resto 3	quociente 295938043; resto 3
2	100 ÷ 7	quociente 14; resto 2	quociente 14; resto 2
3	-100 ÷ -7	quociente 14; resto -2	quociente 14; resto -2
4	-125,5 ÷ 2,5	quociente -50; resto -0,5	quociente -50; resto -0,5
5	7,5 ÷ -2	quociente -3; resto 1,5	quociente -3; resto 1,5
6	1234,5678 ÷ 0,0001	quociente 12345678; resto 0	quociente 12345678; resto 0
7	0 ÷ 7	quociente 0; resto 0	quociente 0; resto 0
8	5 ÷ 10	quociente 0; resto 5	quociente 0; resto 5

O divisor 0 dá a mensagem «Erro: o divisor nao pode ser 0. Tente outra vez.» e o programa volta a pedir o divisor. Um número com mais de 12 algarismos ou com duas vírgulas é recusado («Numero invalido»).

Raiz quadrada

Teste	Entrada	Resultado esperado	Resultado obtido
1	341,7	raiz 18,4; resto 3,14	raiz 18,4; resto 3,14
2	144	raiz 12; resto 0	raiz 12; resto 0
3	2	raiz 1; resto 1	raiz 1; resto 1
4	2,0000	raiz 1,41; resto 0,0119	raiz 1,41; resto 0,0119
5	0,25	raiz 0,5; resto 0	raiz 0,5; resto 0
6	123456789012	raiz 351364; resto 128516	raiz 351364; resto 128516
7	0	raiz 0; resto 0	raiz 0; resto 0

Um número negativo dá a mensagem «Um numero negativo nao tem raiz quadrada real. Tente outra vez.».

Exemplo de execução: divisão

Ecrã do programa da divisão para 125,5 ÷ 2,5 (os números a seguir aos dois pontos são escritos pelo utilizador):

```
DIVISAO INTEIRA COM ARRAYS

Dividendo (ex: -125,5): 125,5
Divisor   (ex: 2,5): 2,5

Sem virgula (mesmas casas decimais): 1255 / 25

Passo 1: numero = 1
  Verificar 1 >= 25 ? Nao -> algarismo do quociente = 0
  quociente = 0 x 10 + 0 = 0

Passo 2: numero = 1 x 10 + 2 = 12
  Verificar 12 >= 25 ? Nao -> algarismo do quociente = 0
  quociente = 0 x 10 + 0 = 0

Passo 3: numero = 12 x 10 + 5 = 125
  Verificar 125 >= 25 ? Sim
  25 x 1 = 25
  25 x 2 = 50
  25 x 3 = 75
  25 x 4 = 100
  25 x 5 = 125
  25 x 6 = 150 > 125 -> usar 5
  resto = 125 - 125 = 0
  quociente = 0 x 10 + 5 = 5

Passo 4: numero = 0 x 10 + 5 = 5
  Verificar 5 >= 25 ? Nao -> algarismo do quociente = 0
  quociente = 5 x 10 + 0 = 50

RESULTADO
Quociente inteiro = 50
Resto = 0,5
```

Exemplo de execução: raiz quadrada

Ecrã do programa da raiz quadrada para 341,7 (exemplo da aula):

```
RAIZ QUADRADA COM ARRAYS

Numero (ex: 341,7): 341,7

Grupos de 2 algarismos: 03 41 , 70
Em cada grupo: maior a com (2 x R x 10 + a) x a <= resto

Grupo 1 (03): resto = 3
  2 x R = 2 x 0 = 0
  1 x 1 = 1
  2 x 2 = 4 > 3 -> usar 1
  resto = 3 - 1 = 2
  R = 0 x 10 + 1 = 1
```

Grupo 2 (41): resto = 2 x 100 + 41 = 241
  2 x R = 2 x 1 = 2
  21 x 1 = 21
  22 x 2 = 44
  23 x 3 = 69
  24 x 4 = 96
  25 x 5 = 125
  26 x 6 = 156
  27 x 7 = 189
  28 x 8 = 224
  29 x 9 = 261 > 241 -> usar 8
  resto = 241 - 224 = 17
  R = 1 x 10 + 8 = 18

Grupo 3 (70): resto = 17 x 100 + 70 = 1770
  2 x R = 2 x 18 = 36
  361 x 1 = 361
  362 x 2 = 724
  363 x 3 = 1089
  364 x 4 = 1456
  365 x 5 = 1825 > 1770 -> usar 4
  resto = 1770 - 1456 = 314
  R = 18 x 10 + 4 = 184

RESULTADO
Raiz quadrada = 18,4
Resto = 3,14

CONCLUSÃO

Os dois programas seguem os algoritmos da aula e mostram no ecrã cada operação parcial: na divisão, o número de cada passo, a verificação, a tabuada do divisor, o resto e o quociente; na raiz, o resto de cada grupo, as tentativas (2R × 10 + a) × a, o novo resto e a raiz parcial. Os resultados dos exemplos da aula (7398451078 ÷ 25 = 295938043, resto 3; raiz de 341,7 = 18,4, resto 3,14) coincidem com os calculados à mão.

Guardar os números em arrays, um algarismo por posição, permitiu trabalhar com números maiores do que os 16 bits dos registos e fazer todas as contas só com adição, subtração e multiplicação, como pede o enunciado. A divisão de números reais reduz-se à divisão inteira igualando as casas decimais, e os sinais tratam-se à parte.

Limitações: cada número tem no máximo 12 algarismos (a constante MAXD pode ser aumentada, aumentando também LEN); o resultado da divisão é o quociente inteiro e o resto; a raiz tem tantas casas decimais quantos os grupos decimais escritos pelo utilizador; a tecla de apagar (Backspace) não é tratada na leitura.

BIBLIOGRAFIA

Hyde, R. The Art of Assembly Language Programming (edição para DOS, 16 bits). Capítulo 5 (Variáveis e estruturas de dados), capítulo 6 (The 80x86 Instruction Set), capítulo 8 (MASM) e capítulo 13 (MS-DOS). Disponível em https://www.phatcode.net/res/223/files/html/Chapter_6/CH06-1.html.

Manso, A., Marques, C., Santos, P., Lopes, L. e Guedes, R. Algorithmi: Bridging the Algorithms to Natural and Programming Languages. Instituto Politécnico de Tomar, 2019.

emu8086: Assembler and Microprocessor Emulator, versão 4.08.

Apontamentos da aula de Arquitetura de Computadores: algoritmos da divisão inteira e da raiz quadrada.

ANEXOS

Cópia dos códigos usados: divisão inteira (Anexo 1) e raiz quadrada (Anexo 2), cada um em versão comentada e sem comentários.

Anexo 1.1: Divisão inteira com arrays, código comentado

```
; ====================================================================
; TAREFA 3 - 1. DIVISAO INTEIRA COM ARRAYS (emu8086)
; Algoritmo da aula (divisao "a mao", algarismo a algarismo):
;   - o dividendo e o divisor sao lidos do teclado e guardados em arrays,
;     um algarismo por posicao; podem ser negativos e/ou ter virgula;
;   - as casas decimais sao igualadas (acrescentando zeros), ficando
;     uma divisao de inteiros;
;   - em cada passo: numero = numero anterior x 10 + algarismo seguinte;
;     faz-se a tabuada do divisor (divisor x 1, x 2, ...) ate passar o
;     numero e usa-se o algarismo anterior;
;     resto = numero - divisor x algarismo;
;     quociente = quociente x 10 + algarismo.
; As contas sobre os arrays usam apenas adicao, subtracao e multiplicacao
; (nao se usa a instrucao div).
; ====================================================================
.model small
.stack 100h

LEN  equ 26              ; tamanho de cada array (algarismos)
MAXD equ 12              ; maximo de algarismos que se podem escrever

.data
    msgTit  db 13,10,'DIVISAO INTEIRA COM ARRAYS',13,10,'$'
    msgA    db 13,10,'Dividendo (ex: -125,5): $'
    msgB    db 13,10,'Divisor   (ex: 2,5): $'
    msgInv  db 13,10,'Numero invalido. Tente outra vez.$'
    msgZero db 13,10,'Erro: o divisor nao pode ser 0. Tente outra vez.$'
    msgSemV db 13,10,13,10,'Sem virgula (mesmas casas decimais): $'
    msgDiv  db ' / $'
    msgPas  db 13,10,13,10,'Passo $'
    msgNum  db ': numero = $'
    msgX10  db ' x 10 + $'
    msgIg   db ' = $'
    msgVer  db 13,10,'  Verificar $'
    msgMaig db ' >= $'
    msgSim  db ' ? Sim$'
    msgNao  db ' ? Nao -> algarismo do quociente = 0$'
    msgTab  db 13,10,'  $'
    msgVez  db ' x $'
    msgMai  db ' > $'
    msgUsa  db ' -> usar $'
    msgRes  db 13,10,'  resto = $'
    msgMen  db ' - $'
    msgQuo  db 13,10,'  quociente = $'
    msgFim  db 13,10,13,10,'RESULTADO$'
    msgQ    db 13,10,'Quociente inteiro = $'
    msgR    db 13,10,'Resto = $'
    BUF     db LEN dup(0)    ; algarismos tal como foram escritos
    NUMA    db LEN dup(0)    ; dividendo (alinhado a direita)
    NUMB    db LEN dup(0)    ; divisor
    PARC    db LEN dup(0)    ; numero do passo atual / resto
    PROD    db LEN dup(0)    ; divisor x k (tabuada)
    PRODOK  db LEN dup(0)    ; maior produto que nao passa o numero
    QUO     db LEN dup(0)    ; quociente
    NDIG    dw ?             ; algarismos lidos
    NDEC    dw ?             ; casas decimais lidas
    SINAL   db ?             ; 1 se o numero lido for negativo
```

```
    SA      db ?            ; sinal do dividendo
    SB      db ?            ; sinal do divisor
    DECA    dw ?            ; casas decimais do dividendo
    DECB    dw ?            ; casas decimais do divisor
    NDECF   dw ?            ; casas decimais depois de igualar
    POS     dw ?            ; posicao do algarismo do dividendo a descer
    PASSO   db ?            ; numero do passo
    DIG     db ?            ; algarismo que desce
    K       db ?            ; algarismo do quociente

.code
main:
    mov  ax, @data
    mov  ds, ax
    lea  dx, msgTit
    call texto

; ---------- ler o dividendo para o array NUMA ----------
le_a:
    lea  dx, msgA
    call texto
    call ler_num            ; BUF, NDIG, NDEC, SINAL; CF = 1 se invalido
    jnc  a_ok
    lea  dx, msgInv
    call texto
    jmp  le_a
a_ok:
    mov  al, SINAL
    mov  SA, al
    mov  ax, NDEC
    mov  DECA, ax
    lea  bx, NUMA
    call para_array         ; copia BUF para NUMA

; ---------- ler o divisor para o array NUMB ----------
le_b:
    lea  dx, msgB
    call texto
    call ler_num
    jnc  b_ok
    lea  dx, msgInv
    call texto
    jmp  le_b
b_ok:
    mov  al, SINAL
    mov  SB, al
    mov  ax, NDEC
    mov  DECB, ax
    lea  bx, NUMB
    call para_array
    lea  si, NUMB
    call e_zero             ; ZF = 1 se o divisor for 0
    jne  b_nz
    lea  dx, msgZero
    call texto
    jmp  le_b
b_nz:

; ---------- igualar as casas decimais (acrescentar zeros) ----------
    mov  ax, DECA
```

```
    mov  cx, DECB
    cmp  ax, cx
    jae  al_b               ; o dividendo tem mais (ou iguais) casas
    sub  cx, ax             ; CX = zeros a acrescentar ao dividendo
    lea  bx, NUMA
al_a:
    mov  al, 0
    call empurra            ; NUMA = NUMA x 10 + 0
    loop al_a
    mov  ax, DECB
    jmp  al_fim
al_b:
    mov  cx, ax
    sub  cx, DECB           ; CX = zeros a acrescentar ao divisor
    cmp  cx, 0
    je   al_fim0            ; nada a acrescentar
    lea  bx, NUMB
al_b2:
    mov  al, 0
    call empurra            ; NUMB = NUMB x 10 + 0
    loop al_b2
al_fim0:
    mov  ax, DECA
al_fim:
    mov  NDECF, ax
    cmp  ax, 0
    je   sem_dec
    lea  dx, msgSemV        ; mostra a divisao de inteiros equivalente
    call texto
    lea  si, NUMA
    xor  cx, cx
    call escreve_arr
    lea  dx, msgDiv
    call texto
    lea  si, NUMB
    xor  cx, cx
    call escreve_arr
sem_dec:

; ---------- divisao algarismo a algarismo ----------
    lea  bx, PARC
    call zera
    lea  bx, QUO
    call zera
    lea  si, NUMA           ; procura o 1.o algarismo diferente de 0
    xor  bx, bx
pd:
    cmp  bx, LEN-1
    jae  pd_ok
    mov  al, [si+bx]
    cmp  al, 0
    jne  pd_ok
    inc  bx
    jmp  pd
pd_ok:
    mov  POS, bx
    mov  al, 0
    mov  PASSO, al

ciclo:
```

```
    mov  al, PASSO
    inc  al
    mov  PASSO, al
    lea  dx, msgPas          ; "Passo n: numero = "
    call texto
    mov  al, PASSO
    call escreve_n8
    lea  dx, msgNum
    call texto
    mov  bx, POS             ; algarismo que desce
    lea  si, NUMA
    mov  al, [si+bx]
    mov  DIG, al
    mov  al, PASSO
    cmp  al, 1
    je   c_prim              ; no 1.o passo nao ha numero anterior
    lea  si, PARC            ; "anterior x 10 + algarismo = "
    xor  cx, cx
    call escreve_arr
    lea  dx, msgX10
    call texto
    mov  al, DIG
    call escreve_dig
    lea  dx, msgIg
    call texto
c_prim:
    lea  bx, PARC
    mov  al, DIG
    call empurra             ; numero = numero anterior x 10 + algarismo
    lea  si, PARC
    xor  cx, cx
    call escreve_arr

; ----- verificar numero >= divisor -----
    lea  dx, msgVer
    call texto
    lea  si, PARC
    xor  cx, cx
    call escreve_arr
    lea  dx, msgMaig
    call texto
    lea  si, NUMB
    xor  cx, cx
    call escreve_arr
    lea  si, PARC
    lea  di, NUMB
    call compara             ; compara numero com divisor
    jae  c_sim
    lea  dx, msgNao
    call texto
    mov  al, 0
    mov  K, al               ; algarismo do quociente = 0
    jmp  c_quoc

; ----- tabuada do divisor: divisor x 1, x 2, ... -----
c_sim:
    lea  dx, msgSim
    call texto
    mov  al, 1
    mov  K, al
```

```
c_tab:
    lea  si, NUMB
    lea  di, PROD
    mov  al, K
    call mul_dig             ; PROD = divisor x K
    lea  dx, msgTab
    call texto
    lea  si, NUMB
    xor  cx, cx
    call escreve_arr
    lea  dx, msgVez
    call texto
    mov  al, K
    call escreve_dig
    lea  dx, msgIg
    call texto
    lea  si, PROD
    xor  cx, cx
    call escreve_arr
    lea  si, PROD
    lea  di, PARC
    call compara             ; o produto passou o numero?
    ja   c_passou
    lea  si, PROD            ; ainda nao passou: guarda-o
    lea  di, PRODOK
    call copia
    mov  al, K
    cmp  al, 9
    je   c_sub               ; 9 e o maior algarismo
    inc  al
    mov  K, al
    jmp  c_tab
c_passou:
    lea  dx, msgMai          ; " > numero -> usar k-1"
    call texto
    lea  si, PARC
    xor  cx, cx
    call escreve_arr
    lea  dx, msgUsa
    call texto
    mov  al, K
    dec  al
    mov  K, al
    call escreve_dig

; ----- resto = numero - divisor x algarismo -----
c_sub:
    lea  dx, msgRes
    call texto
    lea  si, PARC
    xor  cx, cx
    call escreve_arr
    lea  dx, msgMen
    call texto
    lea  si, PRODOK
    xor  cx, cx
    call escreve_arr
    lea  dx, msgIg
    call texto
    lea  si, PARC
    lea  di, PRODOK
```

```
    call subtrai            ; PARC = PARC - PRODOK
    lea  si, PARC
    xor  cx, cx
    call escreve_arr

; ----- quociente = quociente x 10 + algarismo -----
c_quoc:
    lea  dx, msgQuo
    call texto
    lea  si, QUO
    xor  cx, cx
    call escreve_arr
    lea  dx, msgX10
    call texto
    mov  al, K
    call escreve_dig
    lea  dx, msgIg
    call texto
    lea  bx, QUO
    mov  al, K
    call empurra
    lea  si, QUO
    xor  cx, cx
    call escreve_arr
    mov  bx, POS            ; passa ao algarismo seguinte
    inc  bx
    mov  POS, bx
    cmp  bx, LEN-1
    ja   fim_ciclo          ; ja nao ha algarismos
    jmp  ciclo
fim_ciclo:

; ---------- resultado final ----------
    lea  dx, msgFim
    call texto
    lea  dx, msgQ
    call texto
    lea  si, QUO
    call e_zero
    je   q_pos              ; quociente 0 nao leva sinal
    mov  al, SA
    mov  ah, SB
    cmp  al, ah
    je   q_pos              ; sinais iguais: positivo
    call menos
q_pos:
    lea  si, QUO
    xor  cx, cx
    call escreve_arr
    lea  dx, msgR
    call texto
    lea  si, PARC
    xor  cx, cx             ; resto 0: escreve so "0"
    call e_zero
    je   r_pos
    mov  cx, NDECF          ; o resto tem as casas decimais igualadas
    mov  al, SA             ; o resto tem o sinal do dividendo
    cmp  al, 0
    je   r_pos
    call menos
```

```
r_pos:
    lea  si, PARC
    call escreve_arr
    mov  ah, 4Ch             ; termina
    int  21h

; ====================================================================
; ler_num: le um numero do teclado ate ENTER: '-' opcional, algarismos e
; uma virgula ('.' ou ','). Guarda os algarismos em BUF.
; Saida: NDIG, NDEC, SINAL; CF = 1 se o numero for invalido.
; ====================================================================
ler_num proc
    push ax
    push bx
    push cx
    push dx
    push si
    push di
    lea  bx, BUF
    xor  cx, cx              ; CX = algarismos lidos
    xor  di, di              ; DI = casas decimais
    xor  dx, dx              ; DH = 1 se ja leu a virgula, DL = 1 se invalido
    mov  al, 0
    mov  SINAL, al
    mov  ah, 01h
    int  21h                 ; primeiro caracter
    cmp  al, '-'
    jne  ln_ver
    mov  al, 1
    mov  SINAL, al           ; numero negativo
ln_prox:
    mov  ah, 01h
    int  21h
ln_ver:
    cmp  al, 13              ; ENTER?
    je   ln_fim
    cmp  al, ','
    je   ln_virg
    cmp  al, '.'
    je   ln_virg
    sub  al, '0'
    cmp  al, 9
    ja   ln_prox             ; nao e algarismo: ignora
    cmp  cx, MAXD
    jae  ln_mau              ; algarismos a mais
    mov  si, cx
    mov  [bx+si], al         ; BUF[CX] = algarismo
    inc  cx
    cmp  dh, 0
    je   ln_prox
    inc  di                  ; depois da virgula: mais uma casa decimal
    jmp  ln_prox
ln_virg:
    cmp  dh, 0
    jne  ln_mau              ; segunda virgula
    mov  dh, 1
    jmp  ln_prox
ln_mau:
    mov  dl, 1
    jmp  ln_prox
ln_fim:
```

```
    cmp  cx, 0
    je   ln_inv              ; nenhum algarismo
    cmp  dl, 0
    jne  ln_inv
    mov  NDIG, cx
    mov  NDEC, di
    clc
    jmp  ln_sai
ln_inv:
    stc
ln_sai:
    pop  di
    pop  si
    pop  dx
    pop  cx
    pop  bx
    pop  ax
    ret
ler_num endp

; ========================================================================
; para_array: BX = array. Copia os NDIG algarismos de BUF para o array,
; alinhados a direita (array = array x 10 + algarismo, um a um).
; ========================================================================
para_array proc
    push ax
    push cx
    push si
    call zera
    xor  si, si
    mov  cx, NDIG
pa_loop:
    mov  al, BUF[si]
    call empurra
    inc  si
    loop pa_loop
    pop  si
    pop  cx
    pop  ax
    ret
para_array endp

; ========================================================================
; zera: BX = array. Poe todas as posicoes a 0.
; ========================================================================
zera proc
    push ax
    push cx
    push si
    xor  si, si
    xor  al, al
    mov  cx, LEN
z_loop:
    mov  [bx+si], al
    inc  si
    loop z_loop
    pop  si
    pop  cx
    pop  ax
    ret
```

```
zera endp

; ========================================================================
; empurra: BX = array, AL = algarismo. array = array x 10 + AL
; (todos os algarismos andam uma posicao para a esquerda).
; ========================================================================
empurra proc
    push cx
    push dx
    push si
    mov  dl, al
    xor  si, si
    mov  cx, LEN-1
e_loop:
    mov  al, [bx+si+1]
    mov  [bx+si], al
    inc  si
    loop e_loop
    mov  [bx+si], dl         ; ultima posicao = algarismo novo
    mov  al, dl
    pop  si
    pop  dx
    pop  cx
    ret
empurra endp

; ========================================================================
; copia: copia o array SI para o array DI.
; ========================================================================
copia proc
    push ax
    push bx
    push cx
    xor  bx, bx
    mov  cx, LEN
cp_loop:
    mov  al, [si+bx]
    mov  [di+bx], al
    inc  bx
    loop cp_loop
    pop  cx
    pop  bx
    pop  ax
    ret
copia endp

; ========================================================================
; compara: compara o array SI com o array DI, do algarismo mais
; significativo para o menos. Sai com as flags como "cmp SI, DI":
; ja/jae/jb/je funcionam a seguir.
; ========================================================================
compara proc
    push ax
    push cx
    push si
    push di
    mov  cx, LEN
c_loop:
    mov  al, [si]
    cmp  al, [di]
```

```
    jne  c_fim              ; primeira diferenca decide
    inc  si
    inc  di
    loop c_loop
    cmp  al, al             ; iguais: ZF = 1 e CF = 0
c_fim:
    pop  di
    pop  si
    pop  cx
    pop  ax
    ret
compara endp

; ======================================================================
; subtrai: array SI = array SI - array DI (SI >= DI), com "e vai um".
; ======================================================================
subtrai proc
    push ax
    push bx
    mov  bx, LEN-1
    xor  ah, ah             ; AH = emprestimo
s_loop:
    mov  al, [si+bx]
    sub  al, ah
    sub  al, [di+bx]        ; algarismo - emprestimo - algarismo
    mov  ah, 0
    jns  s_ok
    add  al, 10             ; ficou negativo: pede 10 emprestado
    mov  ah, 1
s_ok:
    mov  [si+bx], al
    dec  bx
    jns  s_loop
    pop  bx
    pop  ax
    ret
subtrai endp

; ======================================================================
; mul_dig: array DI = array SI x AL (AL e um algarismo de 0 a 9).
; Em cada posicao: produto = algarismo x AL + transporte; as dezenas
; sao contadas subtraindo 10 (sem usar div).
; ======================================================================
mul_dig proc
    push ax
    push bx
    push dx
    mov  dl, al             ; DL = multiplicador
    xor  dh, dh             ; DH = transporte
    mov  bx, LEN-1
m_loop:
    mov  al, [si+bx]
    mul  dl                 ; AX = algarismo x multiplicador
    add  al, dh             ; + transporte
    xor  dh, dh
m_dez:
    cmp  al, 9
    jbe  m_un               ; ja e menor que 10
    sub  al, 10             ; tira uma dezena
    inc  dh                 ; e conta-a no transporte
```

```
    jmp  m_dez
m_un:
    mov  [di+bx], al
    dec  bx
    jns  m_loop
    pop  dx
    pop  bx
    pop  ax
    ret
mul_dig endp

; ======================================================================
; e_zero: SI = array. Sai com ZF = 1 se todos os algarismos forem 0.
; ======================================================================
e_zero proc
    push ax
    push bx
    push cx
    xor  al, al
    xor  bx, bx
    mov  cx, LEN
ez_loop:
    or   al, [si+bx]
    inc  bx
    loop ez_loop
    cmp  al, 0
    pop  cx
    pop  bx
    pop  ax
    ret
e_zero endp

; ======================================================================
; escreve_arr: escreve o array SI sem os zeros a esquerda.
; CX = casas decimais (escreve a virgula antes delas).
; ======================================================================
escreve_arr proc
    push ax
    push bx
    push dx
    push di
    mov  di, LEN-1
    sub  di, cx              ; DI = posicao das unidades
    xor  bx, bx
ea_proc:
    cmp  bx, di
    jae  ea_loop             ; as unidades escrevem-se sempre
    mov  al, [si+bx]
    cmp  al, 0
    jne  ea_loop
    inc  bx                  ; salta zero a esquerda
    jmp  ea_proc
ea_loop:
    mov  dl, [si+bx]
    add  dl, '0'
    mov  ah, 02h
    int  21h
    cmp  bx, di
    jne  ea_sv
    cmp  cx, 0
```

```
    je   ea_sv
    mov  dl, ','             ; virgula depois das unidades
    mov  ah, 02h
    int  21h
ea_sv:
    inc  bx
    cmp  bx, LEN-1
    jbe  ea_loop
    pop  di
    pop  dx
    pop  bx
    pop  ax
    ret
escreve_arr endp

; ========================================================================
; escreve_dig: escreve o algarismo AL.
; ========================================================================
escreve_dig proc
    push ax
    push dx
    mov  dl, al
    add  dl, '0'
    mov  ah, 02h
    int  21h
    pop  dx
    pop  ax
    ret
escreve_dig endp

; ========================================================================
; escreve_n8: escreve AL (0 a 99); as dezenas contam-se subtraindo 10.
; ========================================================================
escreve_n8 proc
    push ax
    xor  ah, ah              ; AH = dezenas
n8_dez:
    cmp  al, 9
    jbe  n8_esc
    sub  al, 10
    inc  ah
    jmp  n8_dez
n8_esc:
    cmp  ah, 0
    je   n8_un
    push ax
    mov  al, ah
    call escreve_dig         ; dezenas
    pop  ax
n8_un:
    call escreve_dig         ; unidades
    pop  ax
    ret
escreve_n8 endp

; ========================================================================
; menos: escreve o sinal '-'.
; ========================================================================
menos proc
    push ax
```

```
    push dx
    mov  dl, '-'
    mov  ah, 02h
    int  21h
    pop  dx
    pop  ax
    ret
menos endp

; ==========================================================================
; texto: escreve o texto (terminado em '$') cujo endereco esta em DX.
; ==========================================================================
texto proc
    push ax
    mov  ah, 09h
    int  21h
    pop  ax
    ret
texto endp

end main
```

Anexo 1.2: Divisão inteira com arrays, código sem comentários

```
.model small
.stack 100h

LEN  equ 26
MAXD equ 12

.data
    msgTit  db 13,10,'DIVISAO INTEIRA COM ARRAYS',13,10,'$'
    msgA    db 13,10,'Dividendo (ex: -125,5): $'
    msgB    db 13,10,'Divisor   (ex: 2,5): $'
    msgInv  db 13,10,'Numero invalido. Tente outra vez.$'
    msgZero db 13,10,'Erro: o divisor nao pode ser 0. Tente outra vez.$'
    msgSemV db 13,10,13,10,'Sem virgula (mesmas casas decimais): $'
    msgDiv  db ' / $'
    msgPas  db 13,10,13,10,'Passo $'
    msgNum  db ': numero = $'
    msgX10  db ' x 10 + $'
    msgIg   db ' = $'
    msgVer  db 13,10,'  Verificar $'
    msgMaig db ' >= $'
    msgSim  db ' ? Sim$'
    msgNao  db ' ? Nao -> algarismo do quociente = 0$'
    msgTab  db 13,10,'  $'
    msgVez  db ' x $'
    msgMai  db ' > $'
    msgUsa  db ' -> usar $'
    msgRes  db 13,10,'  resto = $'
    msgMen  db ' - $'
    msgQuo  db 13,10,'  quociente = $'
    msgFim  db 13,10,13,10,'RESULTADO$'
    msgQ    db 13,10,'Quociente inteiro = $'
    msgR    db 13,10,'Resto = $'
    BUF     db LEN dup(0)
    NUMA    db LEN dup(0)
    NUMB    db LEN dup(0)
    PARC    db LEN dup(0)
    PROD    db LEN dup(0)
    PRODOK  db LEN dup(0)
    QUO     db LEN dup(0)
    NDIG    dw ?
    NDEC    dw ?
    SINAL   db ?
    SA      db ?
    SB      db ?
    DECA    dw ?
    DECB    dw ?
    NDECF   dw ?
    POS     dw ?
    PASSO   db ?
    DIG     db ?
    K       db ?

.code
main:
    mov  ax, @data
```

```
    mov  ds, ax
    lea  dx, msgTit
    call texto

le_a:
    lea  dx, msgA
    call texto
    call ler_num
    jnc  a_ok
    lea  dx, msgInv
    call texto
    jmp  le_a
a_ok:
    mov  al, SINAL
    mov  SA, al
    mov  ax, NDEC
    mov  DECA, ax
    lea  bx, NUMA
    call para_array

le_b:
    lea  dx, msgB
    call texto
    call ler_num
    jnc  b_ok
    lea  dx, msgInv
    call texto
    jmp  le_b
b_ok:
    mov  al, SINAL
    mov  SB, al
    mov  ax, NDEC
    mov  DECB, ax
    lea  bx, NUMB
    call para_array
    lea  si, NUMB
    call e_zero
    jne  b_nz
    lea  dx, msgZero
    call texto
    jmp  le_b
b_nz:

    mov  ax, DECA
    mov  cx, DECB
    cmp  ax, cx
    jae  al_b
    sub  cx, ax
    lea  bx, NUMA
al_a:
    mov  al, 0
    call empurra
    loop al_a
    mov  ax, DECB
    jmp  al_fim
al_b:
    mov  cx, ax
    sub  cx, DECB
    cmp  cx, 0
    je   al_fim0
```

```
    lea  bx, NUMB
al_b2:
    mov  al, 0
    call empurra
    loop al_b2
al_fim0:
    mov  ax, DECA
al_fim:
    mov  NDECF, ax
    cmp  ax, 0
    je   sem_dec
    lea  dx, msgSemV
    call texto
    lea  si, NUMA
    xor  cx, cx
    call escreve_arr
    lea  dx, msgDiv
    call texto
    lea  si, NUMB
    xor  cx, cx
    call escreve_arr
sem_dec:

    lea  bx, PARC
    call zera
    lea  bx, QUO
    call zera
    lea  si, NUMA
    xor  bx, bx
pd:
    cmp  bx, LEN-1
    jae  pd_ok
    mov  al, [si+bx]
    cmp  al, 0
    jne  pd_ok
    inc  bx
    jmp  pd
pd_ok:
    mov  POS, bx
    mov  al, 0
    mov  PASSO, al

ciclo:
    mov  al, PASSO
    inc  al
    mov  PASSO, al
    lea  dx, msgPas
    call texto
    mov  al, PASSO
    call escreve_n8
    lea  dx, msgNum
    call texto
    mov  bx, POS
    lea  si, NUMA
    mov  al, [si+bx]
    mov  DIG, al
    mov  al, PASSO
    cmp  al, 1
    je   c_prim
    lea  si, PARC
```

```
    xor  cx, cx
    call escreve_arr
    lea  dx, msgX10
    call texto
    mov  al, DIG
    call escreve_dig
    lea  dx, msgIg
    call texto
c_prim:
    lea  bx, PARC
    mov  al, DIG
    call empurra
    lea  si, PARC
    xor  cx, cx
    call escreve_arr

    lea  dx, msgVer
    call texto
    lea  si, PARC
    xor  cx, cx
    call escreve_arr
    lea  dx, msgMaig
    call texto
    lea  si, NUMB
    xor  cx, cx
    call escreve_arr
    lea  si, PARC
    lea  di, NUMB
    call compara
    jae  c_sim
    lea  dx, msgNao
    call texto
    mov  al, 0
    mov  K, al
    jmp  c_quoc

c_sim:
    lea  dx, msgSim
    call texto
    mov  al, 1
    mov  K, al
c_tab:
    lea  si, NUMB
    lea  di, PROD
    mov  al, K
    call mul_dig
    lea  dx, msgTab
    call texto
    lea  si, NUMB
    xor  cx, cx
    call escreve_arr
    lea  dx, msgVez
    call texto
    mov  al, K
    call escreve_dig
    lea  dx, msgIg
    call texto
    lea  si, PROD
    xor  cx, cx
    call escreve_arr
```

```
    lea  si, PROD
    lea  di, PARC
    call compara
    ja   c_passou
    lea  si, PROD
    lea  di, PRODOK
    call copia
    mov  al, K
    cmp  al, 9
    je   c_sub
    inc  al
    mov  K, al
    jmp  c_tab
c_passou:
    lea  dx, msgMai
    call texto
    lea  si, PARC
    xor  cx, cx
    call escreve_arr
    lea  dx, msgUsa
    call texto
    mov  al, K
    dec  al
    mov  K, al
    call escreve_dig

c_sub:
    lea  dx, msgRes
    call texto
    lea  si, PARC
    xor  cx, cx
    call escreve_arr
    lea  dx, msgMen
    call texto
    lea  si, PRODOK
    xor  cx, cx
    call escreve_arr
    lea  dx, msgIg
    call texto
    lea  si, PARC
    lea  di, PRODOK
    call subtrai
    lea  si, PARC
    xor  cx, cx
    call escreve_arr

c_quoc:
    lea  dx, msgQuo
    call texto
    lea  si, QUO
    xor  cx, cx
    call escreve_arr
    lea  dx, msgX10
    call texto
    mov  al, K
    call escreve_dig
    lea  dx, msgIg
    call texto
    lea  bx, QUO
    mov  al, K
```

```
    call empurra
    lea  si, QUO
    xor  cx, cx
    call escreve_arr
    mov  bx, POS
    inc  bx
    mov  POS, bx
    cmp  bx, LEN-1
    ja   fim_ciclo
    jmp  ciclo
fim_ciclo:

    lea  dx, msgFim
    call texto
    lea  dx, msgQ
    call texto
    lea  si, QUO
    call e_zero
    je   q_pos
    mov  al, SA
    mov  ah, SB
    cmp  al, ah
    je   q_pos
    call menos
q_pos:
    lea  si, QUO
    xor  cx, cx
    call escreve_arr
    lea  dx, msgR
    call texto
    lea  si, PARC
    xor  cx, cx
    call e_zero
    je   r_pos
    mov  cx, NDECF
    mov  al, SA
    cmp  al, 0
    je   r_pos
    call menos
r_pos:
    lea  si, PARC
    call escreve_arr
    mov  ah, 4Ch
    int  21h

ler_num proc
    push ax
    push bx
    push cx
    push dx
    push si
    push di
    lea  bx, BUF
    xor  cx, cx
    xor  di, di
    xor  dx, dx
    mov  al, 0
    mov  SINAL, al
    mov  ah, 01h
    int  21h
```

```
    cmp  al, '-'
    jne  ln_ver
    mov  al, 1
    mov  SINAL, al
ln_prox:
    mov  ah, 01h
    int  21h
ln_ver:
    cmp  al, 13
    je   ln_fim
    cmp  al, ','
    je   ln_virg
    cmp  al, '.'
    je   ln_virg
    sub  al, '0'
    cmp  al, 9
    ja   ln_prox
    cmp  cx, MAXD
    jae  ln_mau
    mov  si, cx
    mov  [bx+si], al
    inc  cx
    cmp  dh, 0
    je   ln_prox
    inc  di
    jmp  ln_prox
ln_virg:
    cmp  dh, 0
    jne  ln_mau
    mov  dh, 1
    jmp  ln_prox
ln_mau:
    mov  dl, 1
    jmp  ln_prox
ln_fim:
    cmp  cx, 0
    je   ln_inv
    cmp  dl, 0
    jne  ln_inv
    mov  NDIG, cx
    mov  NDEC, di
    clc
    jmp  ln_sai
ln_inv:
    stc
ln_sai:
    pop  di
    pop  si
    pop  dx
    pop  cx
    pop  bx
    pop  ax
    ret
ler_num endp

para_array proc
    push ax
    push cx
    push si
    call zera
    xor  si, si
```

```
    mov  cx, NDIG
pa_loop:
    mov  al, BUF[si]
    call empurra
    inc  si
    loop pa_loop
    pop  si
    pop  cx
    pop  ax
    ret
para_array endp

zera proc
    push ax
    push cx
    push si
    xor  si, si
    xor  al, al
    mov  cx, LEN
z_loop:
    mov  [bx+si], al
    inc  si
    loop z_loop
    pop  si
    pop  cx
    pop  ax
    ret
zera endp

empurra proc
    push cx
    push dx
    push si
    mov  dl, al
    xor  si, si
    mov  cx, LEN-1
e_loop:
    mov  al, [bx+si+1]
    mov  [bx+si], al
    inc  si
    loop e_loop
    mov  [bx+si], dl
    mov  al, dl
    pop  si
    pop  dx
    pop  cx
    ret
empurra endp

copia proc
    push ax
    push bx
    push cx
    xor  bx, bx
    mov  cx, LEN
cp_loop:
    mov  al, [si+bx]
    mov  [di+bx], al
    inc  bx
    loop cp_loop
```

```
    pop  cx
    pop  bx
    pop  ax
    ret
copia endp

compara proc
    push ax
    push cx
    push si
    push di
    mov  cx, LEN
c_loop:
    mov  al, [si]
    cmp  al, [di]
    jne  c_fim
    inc  si
    inc  di
    loop c_loop
    cmp  al, al
c_fim:
    pop  di
    pop  si
    pop  cx
    pop  ax
    ret
compara endp

subtrai proc
    push ax
    push bx
    mov  bx, LEN-1
    xor  ah, ah
s_loop:
    mov  al, [si+bx]
    sub  al, ah
    sub  al, [di+bx]
    mov  ah, 0
    jns  s_ok
    add  al, 10
    mov  ah, 1
s_ok:
    mov  [si+bx], al
    dec  bx
    jns  s_loop
    pop  bx
    pop  ax
    ret
subtrai endp

mul_dig proc
    push ax
    push bx
    push dx
    mov  dl, al
    xor  dh, dh
    mov  bx, LEN-1
m_loop:
    mov  al, [si+bx]
    mul  dl
```

```
    add  al, dh
    xor  dh, dh
m_dez:
    cmp  al, 9
    jbe  m_un
    sub  al, 10
    inc  dh
    jmp  m_dez
m_un:
    mov  [di+bx], al
    dec  bx
    jns  m_loop
    pop  dx
    pop  bx
    pop  ax
    ret
mul_dig endp

e_zero proc
    push ax
    push bx
    push cx
    xor  al, al
    xor  bx, bx
    mov  cx, LEN
ez_loop:
    or   al, [si+bx]
    inc  bx
    loop ez_loop
    cmp  al, 0
    pop  cx
    pop  bx
    pop  ax
    ret
e_zero endp

escreve_arr proc
    push ax
    push bx
    push dx
    push di
    mov  di, LEN-1
    sub  di, cx
    xor  bx, bx
ea_proc:
    cmp  bx, di
    jae  ea_loop
    mov  al, [si+bx]
    cmp  al, 0
    jne  ea_loop
    inc  bx
    jmp  ea_proc
ea_loop:
    mov  dl, [si+bx]
    add  dl, '0'
    mov  ah, 02h
    int  21h
    cmp  bx, di
    jne  ea_sv
    cmp  cx, 0
```

```
    je   ea_sv
    mov  dl, ','
    mov  ah, 02h
    int  21h
ea_sv:
    inc  bx
    cmp  bx, LEN-1
    jbe  ea_loop
    pop  di
    pop  dx
    pop  bx
    pop  ax
    ret
escreve_arr endp

escreve_dig proc
    push ax
    push dx
    mov  dl, al
    add  dl, '0'
    mov  ah, 02h
    int  21h
    pop  dx
    pop  ax
    ret
escreve_dig endp

escreve_n8 proc
    push ax
    xor  ah, ah
n8_dez:
    cmp  al, 9
    jbe  n8_esc
    sub  al, 10
    inc  ah
    jmp  n8_dez
n8_esc:
    cmp  ah, 0
    je   n8_un
    push ax
    mov  al, ah
    call escreve_dig
    pop  ax
n8_un:
    call escreve_dig
    pop  ax
    ret
escreve_n8 endp

menos proc
    push ax
    push dx
    mov  dl, '-'
    mov  ah, 02h
    int  21h
    pop  dx
    pop  ax
    ret
menos endp
```

```
texto proc
    push ax
    mov  ah, 09h
    int  21h
    pop  ax
    ret
texto endp

end main
```

Anexo 2.1: Raiz quadrada com arrays, código comentado

```
; ================================================================
; TAREFA 3 - 2. RAIZ QUADRADA COM ARRAYS (emu8086)
; Algoritmo da aula (raiz quadrada "a mao", algarismo a algarismo):
;   - o numero e lido do teclado e guardado num array, um algarismo por
;     posicao; pode ter virgula;
;   - os algarismos agrupam-se 2 a 2 a partir da virgula (para a esquerda
;     na parte inteira e para a direita na parte decimal); se faltar um
;     algarismo acrescenta-se um 0;
;   - para cada grupo: resto = resto x 100 + grupo; procura-se o maior
;     algarismo a tal que (2 x R x 10 + a) x a <= resto (R = raiz ja
;     calculada); resto = resto - (2 x R x 10 + a) x a; R = R x 10 + a.
; As contas sobre os arrays usam apenas adicao, subtracao e multiplicacao.
; ================================================================
.model small
.stack 100h

LEN  equ 26              ; tamanho de cada array (algarismos)
MAXD equ 12              ; maximo de algarismos que se podem escrever

.data
    msgTit  db 13,10,'RAIZ QUADRADA COM ARRAYS',13,10,'$'
    msgN    db 13,10,'Numero (ex: 341,7): $'
    msgInv  db 13,10,'Numero invalido. Tente outra vez.$'
    msgNeg  db 13,10,'Um numero negativo nao tem raiz quadrada real. Tente outra vez.$'
    msgGr   db 13,10,13,10,'Grupos de 2 algarismos: $'
    msgVg   db ', $'
    msgEsp  db ' $'
    msgReg  db 13,10,'Em cada grupo: maior a com (2 x R x 10 + a) x a <= resto$'
    msgGrp  db 13,10,13,10,'Grupo $'
    msgAbre db ' ($'
    msgFech db '): resto = $'
    msgX100 db ' x 100 + $'
    msgIg   db ' = $'
    msgDob  db 13,10,'  2 x R = 2 x $'
    msgTab  db 13,10,'  $'
    msgVez  db ' x $'
    msgMai  db ' > $'
    msgUsa  db ' -> usar $'
    msgRes  db 13,10,'  resto = $'
    msgMen  db ' - $'
    msgRz   db 13,10,'  R = $'
    msgX10  db ' x 10 + $'
    msgFim  db 13,10,13,10,'RESULTADO$'
    msgRQ   db 13,10,'Raiz quadrada = $'
    msgR    db 13,10,'Resto = $'
    BUF     db LEN dup(0)    ; algarismos tal como foram escritos
    GBUF    db LEN dup(0)    ; algarismos agrupados 2 a 2 (com zeros)
    RAIZ    db LEN dup(0)    ; raiz ja calculada (R)
    RESTO   db LEN dup(0)    ; resto
    DOBRO   db LEN dup(0)    ; 2 x R
    TENT    db LEN dup(0)    ; 2 x R x 10 + a
    PROD    db LEN dup(0)    ; (2 x R x 10 + a) x a
    PRODOK  db LEN dup(0)    ; maior produto que nao passa o resto
    NDIG    dw ?             ; algarismos lidos
    NDEC    dw ?             ; casas decimais lidas
    SINAL   db ?             ; 1 se o numero lido for negativo
    NG      dw ?             ; algarismos em GBUF (par)
```

```
    NGI     dw ?            ; algarismos da parte inteira em GBUF
    NDECR   dw ?            ; casas decimais da raiz (grupos decimais)
    NDECR2  dw ?            ; casas decimais do resto (2 x NDECR)
    IDX     dw ?            ; posicao do grupo em GBUF
    PASSO   db ?            ; numero do grupo
    D1      db ?            ; 1.o algarismo do grupo
    D2      db ?            ; 2.o algarismo do grupo
    A       db ?            ; algarismo a experimentar
    MELHOR  db ?            ; maior a que serve

.code
main:
    mov  ax, @data
    mov  ds, ax
    lea  dx, msgTit
    call texto

; ---------- ler o numero ----------
le:
    lea  dx, msgN
    call texto
    call ler_num            ; BUF, NDIG, NDEC, SINAL; CF = 1 se invalido
    jnc  le_ok
    lea  dx, msgInv
    call texto
    jmp  le
le_ok:
    mov  al, SINAL
    cmp  al, 0
    je   le_pos
    lea  dx, msgNeg         ; negativo: nao tem raiz real
    call texto
    jmp  le
le_pos:

; ---------- agrupar 2 a 2 a partir da virgula ----------
    lea  bx, GBUF
    call zera
    xor  di, di             ; DI = algarismos ja postos em GBUF
    mov  ax, NDIG
    sub  ax, NDEC           ; AX = algarismos da parte inteira
    test al, 1
    jz   g_ipar
    mov  al, 0              ; parte inteira impar: 0 a esquerda
    mov  [bx+di], al
    inc  di
g_ipar:
    xor  si, si
    mov  cx, NDIG
g_cp:
    mov  al, BUF[si]        ; copia os algarismos lidos
    mov  [bx+di], al
    inc  si
    inc  di
    loop g_cp
    mov  ax, NDEC
    test al, 1
    jz   g_dpar
    mov  al, 0              ; parte decimal impar: 0 a direita
    mov  [bx+di], al
```

```
    inc  di
    mov  ax, NDEC
    inc  ax                  ; AX = casas decimais (ja par)
g_dpar:
    mov  NG, di
    mov  dx, di
    sub  dx, ax
    mov  NGI, dx             ; algarismos da parte inteira (par)
    mov  NDECR2, ax          ; o resto tem estas casas decimais
    xor  cx, cx              ; NDECR = AX / 2, contando subtracoes de 2
g_meio:
    cmp  ax, 0
    je   g_meio_ok
    sub  ax, 2
    inc  cx
    jmp  g_meio
g_meio_ok:
    mov  NDECR, cx

; ----- mostra os grupos -----
    lea  dx, msgGr
    call texto
    xor  si, si
gp:
    cmp  si, NG
    jae  gp_fim
    cmp  si, NGI
    jne  gp_d
    cmp  si, 0
    je   gp_d                ; sem parte inteira nao ha virgula antes
    lea  dx, msgVg           ; virgula entre a parte inteira e a decimal
    call texto
gp_d:
    mov  al, GBUF[si]
    call escreve_dig
    inc  si
    mov  al, GBUF[si]
    call escreve_dig
    inc  si
    lea  dx, msgEsp
    call texto
    jmp  gp
gp_fim:
    lea  dx, msgReg
    call texto

; ---------- calculo grupo a grupo ----------
    lea  bx, RAIZ
    call zera
    lea  bx, RESTO
    call zera
    mov  ax, 0
    mov  IDX, ax
    mov  al, 0
    mov  PASSO, al

gciclo:
    mov  al, PASSO
    inc  al
    mov  PASSO, al
```

```
    mov  si, IDX             ; os dois algarismos do grupo
    mov  al, GBUF[si]
    mov  D1, al
    inc  si
    mov  al, GBUF[si]
    mov  D2, al
    lea  dx, msgGrp          ; "Grupo n (d1d2): resto = "
    call texto
    mov  al, PASSO
    call escreve_n8
    lea  dx, msgAbre
    call texto
    mov  al, D1
    call escreve_dig
    mov  al, D2
    call escreve_dig
    lea  dx, msgFech
    call texto
    mov  al, PASSO
    cmp  al, 1
    je   g_prim              ; no 1.o grupo nao ha resto anterior
    lea  si, RESTO           ; "resto anterior x 100 + grupo = "
    xor  cx, cx
    call escreve_arr
    lea  dx, msgX100
    call texto
    mov  al, D1
    call escreve_dig
    mov  al, D2
    call escreve_dig
    lea  dx, msgIg
    call texto
g_prim:
    lea  bx, RESTO
    mov  al, D1
    call empurra             ; resto = resto x 100 + grupo
    mov  al, D2
    call empurra
    lea  si, RESTO
    xor  cx, cx
    call escreve_arr

; ----- 2 x R -----
    lea  si, RAIZ
    lea  di, DOBRO
    mov  al, 2
    call mul_dig             ; DOBRO = 2 x R
    lea  dx, msgDob
    call texto
    lea  si, RAIZ
    xor  cx, cx
    call escreve_arr
    lea  dx, msgIg
    call texto
    lea  si, DOBRO
    xor  cx, cx
    call escreve_arr

; ----- experimentar a = 1, 2, ... : (2 x R x 10 + a) x a -----
    lea  bx, PRODOK
```

```
    call zera
    mov  al, 0
    mov  MELHOR, al
    mov  al, 1
    mov  A, al
t_loop:
    lea  si, DOBRO
    lea  di, TENT
    call copia
    lea  bx, TENT
    mov  al, A
    call empurra            ; TENT = 2 x R x 10 + a
    lea  si, TENT
    lea  di, PROD
    mov  al, A
    call mul_dig            ; PROD = TENT x a
    lea  dx, msgTab
    call texto
    lea  si, TENT
    xor  cx, cx
    call escreve_arr
    lea  dx, msgVez
    call texto
    mov  al, A
    call escreve_dig
    lea  dx, msgIg
    call texto
    lea  si, PROD
    xor  cx, cx
    call escreve_arr
    lea  si, PROD
    lea  di, RESTO
    call compara            ; passou o resto?
    ja   t_pass
    lea  si, PROD           ; ainda serve: guarda-o
    lea  di, PRODOK
    call copia
    mov  al, A
    mov  MELHOR, al
    cmp  al, 9
    je   t_fim              ; 9 e o maior algarismo
    inc  al
    mov  A, al
    jmp  t_loop
t_pass:
    lea  dx, msgMai         ; " > resto -> usar a anterior"
    call texto
    lea  si, RESTO
    xor  cx, cx
    call escreve_arr
    lea  dx, msgUsa
    call texto
    mov  al, MELHOR
    call escreve_dig
t_fim:

; ----- resto = resto - produto escolhido -----
    lea  dx, msgRes
    call texto
    lea  si, RESTO
    xor  cx, cx
```

```
    call escreve_arr
    lea  dx, msgMen
    call texto
    lea  si, PRODOK
    xor  cx, cx
    call escreve_arr
    lea  dx, msgIg
    call texto
    lea  si, RESTO
    lea  di, PRODOK
    call subtrai            ; RESTO = RESTO - PRODOK
    lea  si, RESTO
    xor  cx, cx
    call escreve_arr

; ----- R = R x 10 + a -----
    lea  dx, msgRz
    call texto
    lea  si, RAIZ
    xor  cx, cx
    call escreve_arr
    lea  dx, msgX10
    call texto
    mov  al, MELHOR
    call escreve_dig
    lea  dx, msgIg
    call texto
    lea  bx, RAIZ
    mov  al, MELHOR
    call empurra
    lea  si, RAIZ
    xor  cx, cx
    call escreve_arr
    mov  ax, IDX            ; grupo seguinte
    add  ax, 2
    mov  IDX, ax
    cmp  ax, NG
    jae  g_fim              ; ja nao ha grupos
    jmp  gciclo
g_fim:

; ---------- resultado final ----------
    lea  dx, msgFim
    call texto
    lea  dx, msgRQ
    call texto
    lea  si, RAIZ
    mov  cx, NDECR          ; uma casa decimal por grupo decimal
    call escreve_arr
    lea  dx, msgR
    call texto
    lea  si, RESTO
    xor  cx, cx             ; resto 0: escreve so "0"
    call e_zero
    je   r_esc
    mov  cx, NDECR2         ; o resto tem duas casas por grupo decimal
r_esc:
    call escreve_arr
    mov  ah, 4Ch            ; termina
    int  21h
```

```
; ======================================================================
; ler_num: le um numero do teclado ate ENTER: '-' opcional, algarismos e
; uma virgula ('.' ou ','). Guarda os algarismos em BUF.
; Saida: NDIG, NDEC, SINAL; CF = 1 se o numero for invalido.
; ======================================================================
ler_num proc
    push ax
    push bx
    push cx
    push dx
    push si
    push di
    lea  bx, BUF
    xor  cx, cx              ; CX = algarismos lidos
    xor  di, di              ; DI = casas decimais
    xor  dx, dx              ; DH = 1 se ja leu a virgula, DL = 1 se invalido
    mov  al, 0
    mov  SINAL, al
    mov  ah, 01h
    int  21h                 ; primeiro caracter
    cmp  al, '-'
    jne  ln_ver
    mov  al, 1
    mov  SINAL, al           ; numero negativo
ln_prox:
    mov  ah, 01h
    int  21h
ln_ver:
    cmp  al, 13              ; ENTER?
    je   ln_fim
    cmp  al, ','
    je   ln_virg
    cmp  al, '.'
    je   ln_virg
    sub  al, '0'
    cmp  al, 9
    ja   ln_prox             ; nao e algarismo: ignora
    cmp  cx, MAXD
    jae  ln_mau              ; algarismos a mais
    mov  si, cx
    mov  [bx+si], al         ; BUF[CX] = algarismo
    inc  cx
    cmp  dh, 0
    je   ln_prox
    inc  di                  ; depois da virgula: mais uma casa decimal
    jmp  ln_prox
ln_virg:
    cmp  dh, 0
    jne  ln_mau              ; segunda virgula
    mov  dh, 1
    jmp  ln_prox
ln_mau:
    mov  dl, 1
    jmp  ln_prox
ln_fim:
    cmp  cx, 0
    je   ln_inv              ; nenhum algarismo
    cmp  dl, 0
    jne  ln_inv
    mov  NDIG, cx
```

```
    mov  NDEC, di
    clc
    jmp  ln_sai
ln_inv:
    stc
ln_sai:
    pop  di
    pop  si
    pop  dx
    pop  cx
    pop  bx
    pop  ax
    ret
ler_num endp

; ========================================================================
; para_array: BX = array. Copia os NDIG algarismos de BUF para o array,
; alinhados a direita (array = array x 10 + algarismo, um a um).
; ========================================================================
para_array proc
    push ax
    push cx
    push si
    call zera
    xor  si, si
    mov  cx, NDIG
pa_loop:
    mov  al, BUF[si]
    call empurra
    inc  si
    loop pa_loop
    pop  si
    pop  cx
    pop  ax
    ret
para_array endp

; ========================================================================
; zera: BX = array. Poe todas as posicoes a 0.
; ========================================================================
zera proc
    push ax
    push cx
    push si
    xor  si, si
    xor  al, al
    mov  cx, LEN
z_loop:
    mov  [bx+si], al
    inc  si
    loop z_loop
    pop  si
    pop  cx
    pop  ax
    ret
zera endp

; ========================================================================
; empurra: BX = array, AL = algarismo. array = array x 10 + AL
; (todos os algarismos andam uma posicao para a esquerda).
```

```
; ====================================================================
empurra proc
    push cx
    push dx
    push si
    mov  dl, al
    xor  si, si
    mov  cx, LEN-1
e_loop:
    mov  al, [bx+si+1]
    mov  [bx+si], al
    inc  si
    loop e_loop
    mov  [bx+si], dl         ; ultima posicao = algarismo novo
    mov  al, dl
    pop  si
    pop  dx
    pop  cx
    ret
empurra endp

; ====================================================================
; copia: copia o array SI para o array DI.
; ====================================================================
copia proc
    push ax
    push bx
    push cx
    xor  bx, bx
    mov  cx, LEN
cp_loop:
    mov  al, [si+bx]
    mov  [di+bx], al
    inc  bx
    loop cp_loop
    pop  cx
    pop  bx
    pop  ax
    ret
copia endp

; ====================================================================
; compara: compara o array SI com o array DI, do algarismo mais
; significativo para o menos. Sai com as flags como "cmp SI, DI":
; ja/jae/jb/je funcionam a seguir.
; ====================================================================
compara proc
    push ax
    push cx
    push si
    push di
    mov  cx, LEN
c_loop:
    mov  al, [si]
    cmp  al, [di]
    jne  c_fim               ; primeira diferenca decide
    inc  si
    inc  di
    loop c_loop
    cmp  al, al              ; iguais: ZF = 1 e CF = 0
```

```
c_fim:
    pop  di
    pop  si
    pop  cx
    pop  ax
    ret
compara endp

; ========================================================================
; subtrai: array SI = array SI - array DI (SI >= DI), com "e vai um".
; ========================================================================
subtrai proc
    push ax
    push bx
    mov  bx, LEN-1
    xor  ah, ah              ; AH = emprestimo
s_loop:
    mov  al, [si+bx]
    sub  al, ah
    sub  al, [di+bx]         ; algarismo - emprestimo - algarismo
    mov  ah, 0
    jns  s_ok
    add  al, 10              ; ficou negativo: pede 10 emprestado
    mov  ah, 1
s_ok:
    mov  [si+bx], al
    dec  bx
    jns  s_loop
    pop  bx
    pop  ax
    ret
subtrai endp

; ========================================================================
; mul_dig: array DI = array SI x AL (AL e um algarismo de 0 a 9).
; Em cada posicao: produto = algarismo x AL + transporte; as dezenas
; sao contadas subtraindo 10 (sem usar div).
; ========================================================================
mul_dig proc
    push ax
    push bx
    push dx
    mov  dl, al              ; DL = multiplicador
    xor  dh, dh              ; DH = transporte
    mov  bx, LEN-1
m_loop:
    mov  al, [si+bx]
    mul  dl                  ; AX = algarismo x multiplicador
    add  al, dh              ; + transporte
    xor  dh, dh
m_dez:
    cmp  al, 9
    jbe  m_un                ; ja e menor que 10
    sub  al, 10              ; tira uma dezena
    inc  dh                  ; e conta-a no transporte
    jmp  m_dez
m_un:
    mov  [di+bx], al
    dec  bx
    jns  m_loop
```

```
    pop  dx
    pop  bx
    pop  ax
    ret
mul_dig endp

; ========================================================================
; e_zero: SI = array. Sai com ZF = 1 se todos os algarismos forem 0.
; ========================================================================
e_zero proc
    push ax
    push bx
    push cx
    xor  al, al
    xor  bx, bx
    mov  cx, LEN
ez_loop:
    or   al, [si+bx]
    inc  bx
    loop ez_loop
    cmp  al, 0
    pop  cx
    pop  bx
    pop  ax
    ret
e_zero endp

; ========================================================================
; escreve_arr: escreve o array SI sem os zeros a esquerda.
; CX = casas decimais (escreve a virgula antes delas).
; ========================================================================
escreve_arr proc
    push ax
    push bx
    push dx
    push di
    mov  di, LEN-1
    sub  di, cx              ; DI = posicao das unidades
    xor  bx, bx
ea_proc:
    cmp  bx, di
    jae  ea_loop             ; as unidades escrevem-se sempre
    mov  al, [si+bx]
    cmp  al, 0
    jne  ea_loop
    inc  bx                  ; salta zero a esquerda
    jmp  ea_proc
ea_loop:
    mov  dl, [si+bx]
    add  dl, '0'
    mov  ah, 02h
    int  21h
    cmp  bx, di
    jne  ea_sv
    cmp  cx, 0
    je   ea_sv
    mov  dl, ','             ; virgula depois das unidades
    mov  ah, 02h
    int  21h
ea_sv:
```

```
    inc  bx
    cmp  bx, LEN-1
    jbe  ea_loop
    pop  di
    pop  dx
    pop  bx
    pop  ax
    ret
escreve_arr endp

; ========================================================================
; escreve_dig: escreve o algarismo AL.
; ========================================================================
escreve_dig proc
    push ax
    push dx
    mov  dl, al
    add  dl, '0'
    mov  ah, 02h
    int  21h
    pop  dx
    pop  ax
    ret
escreve_dig endp

; ========================================================================
; escreve_n8: escreve AL (0 a 99); as dezenas contam-se subtraindo 10.
; ========================================================================
escreve_n8 proc
    push ax
    xor  ah, ah              ; AH = dezenas
n8_dez:
    cmp  al, 9
    jbe  n8_esc
    sub  al, 10
    inc  ah
    jmp  n8_dez
n8_esc:
    cmp  ah, 0
    je   n8_un
    push ax
    mov  al, ah
    call escreve_dig         ; dezenas
    pop  ax
n8_un:
    call escreve_dig         ; unidades
    pop  ax
    ret
escreve_n8 endp

; ========================================================================
; menos: escreve o sinal '-'.
; ========================================================================
menos proc
    push ax
    push dx
    mov  dl, '-'
    mov  ah, 02h
    int  21h
    pop  dx
```

```
    pop  ax
    ret
menos endp

; ==========================================================================
; texto: escreve o texto (terminado em '$') cujo endereco esta em DX.
; ==========================================================================
texto proc
    push ax
    mov  ah, 09h
    int  21h
    pop  ax
    ret
texto endp

end main
```

Anexo 2.2: Raiz quadrada com arrays, código sem comentários

```
.model small
.stack 100h

LEN  equ 26
MAXD equ 12

.data
    msgTit  db 13,10,'RAIZ QUADRADA COM ARRAYS',13,10,'$'
    msgN    db 13,10,'Numero (ex: 341,7): $'
    msgInv  db 13,10,'Numero invalido. Tente outra vez.$'
    msgNeg  db 13,10,'Um numero negativo nao tem raiz quadrada real. Tente outra vez.$'
    msgGr   db 13,10,13,10,'Grupos de 2 algarismos: $'
    msgVg   db ', $'
    msgEsp  db ' $'
    msgReg  db 13,10,'Em cada grupo: maior a com (2 x R x 10 + a) x a <= resto$'
    msgGrp  db 13,10,13,10,'Grupo $'
    msgAbre db ' ($'
    msgFech db '): resto = $'
    msgX100 db ' x 100 + $'
    msgIg   db ' = $'
    msgDob  db 13,10,'  2 x R = 2 x $'
    msgTab  db 13,10,'  $'
    msgVez  db ' x $'
    msgMai  db ' > $'
    msgUsa  db ' -> usar $'
    msgRes  db 13,10,'  resto = $'
    msgMen  db ' - $'
    msgRz   db 13,10,'  R = $'
    msgX10  db ' x 10 + $'
    msgFim  db 13,10,13,10,'RESULTADO$'
    msgRQ   db 13,10,'Raiz quadrada = $'
    msgR    db 13,10,'Resto = $'
    BUF     db LEN dup(0)
    GBUF    db LEN dup(0)
    RAIZ    db LEN dup(0)
    RESTO   db LEN dup(0)
    DOBRO   db LEN dup(0)
    TENT    db LEN dup(0)
    PROD    db LEN dup(0)
    PRODOK  db LEN dup(0)
    NDIG    dw ?
    NDEC    dw ?
    SINAL   db ?
    NG      dw ?
    NGI     dw ?
    NDECR   dw ?
    NDECR2  dw ?
    IDX     dw ?
    PASSO   db ?
    D1      db ?
    D2      db ?
    A       db ?
    MELHOR  db ?

.code
```

```
main:
    mov  ax, @data
    mov  ds, ax
    lea  dx, msgTit
    call texto

le:
    lea  dx, msgN
    call texto
    call ler_num
    jnc  le_ok
    lea  dx, msgInv
    call texto
    jmp  le
le_ok:
    mov  al, SINAL
    cmp  al, 0
    je   le_pos
    lea  dx, msgNeg
    call texto
    jmp  le
le_pos:

    lea  bx, GBUF
    call zera
    xor  di, di
    mov  ax, NDIG
    sub  ax, NDEC
    test al, 1
    jz   g_ipar
    mov  al, 0
    mov  [bx+di], al
    inc  di
g_ipar:
    xor  si, si
    mov  cx, NDIG
g_cp:
    mov  al, BUF[si]
    mov  [bx+di], al
    inc  si
    inc  di
    loop g_cp
    mov  ax, NDEC
    test al, 1
    jz   g_dpar
    mov  al, 0
    mov  [bx+di], al
    inc  di
    mov  ax, NDEC
    inc  ax
g_dpar:
    mov  NG, di
    mov  dx, di
    sub  dx, ax
    mov  NGI, dx
    mov  NDECR2, ax
    xor  cx, cx
g_meio:
    cmp  ax, 0
    je   g_meio_ok
```

```
    sub  ax, 2
    inc  cx
    jmp  g_meio
g_meio_ok:
    mov  NDECR, cx

    lea  dx, msgGr
    call texto
    xor  si, si
gp:
    cmp  si, NG
    jae  gp_fim
    cmp  si, NGI
    jne  gp_d
    cmp  si, 0
    je   gp_d
    lea  dx, msgVg
    call texto
gp_d:
    mov  al, GBUF[si]
    call escreve_dig
    inc  si
    mov  al, GBUF[si]
    call escreve_dig
    inc  si
    lea  dx, msgEsp
    call texto
    jmp  gp
gp_fim:
    lea  dx, msgReg
    call texto

    lea  bx, RAIZ
    call zera
    lea  bx, RESTO
    call zera
    mov  ax, 0
    mov  IDX, ax
    mov  al, 0
    mov  PASSO, al

gciclo:
    mov  al, PASSO
    inc  al
    mov  PASSO, al
    mov  si, IDX
    mov  al, GBUF[si]
    mov  D1, al
    inc  si
    mov  al, GBUF[si]
    mov  D2, al
    lea  dx, msgGrp
    call texto
    mov  al, PASSO
    call escreve_n8
    lea  dx, msgAbre
    call texto
    mov  al, D1
    call escreve_dig
    mov  al, D2
```

```
    call escreve_dig
    lea  dx, msgFech
    call texto
    mov  al, PASSO
    cmp  al, 1
    je   g_prim
    lea  si, RESTO
    xor  cx, cx
    call escreve_arr
    lea  dx, msgX100
    call texto
    mov  al, D1
    call escreve_dig
    mov  al, D2
    call escreve_dig
    lea  dx, msgIg
    call texto
g_prim:
    lea  bx, RESTO
    mov  al, D1
    call empurra
    mov  al, D2
    call empurra
    lea  si, RESTO
    xor  cx, cx
    call escreve_arr

    lea  si, RAIZ
    lea  di, DOBRO
    mov  al, 2
    call mul_dig
    lea  dx, msgDob
    call texto
    lea  si, RAIZ
    xor  cx, cx
    call escreve_arr
    lea  dx, msgIg
    call texto
    lea  si, DOBRO
    xor  cx, cx
    call escreve_arr

    lea  bx, PRODOK
    call zera
    mov  al, 0
    mov  MELHOR, al
    mov  al, 1
    mov  A, al
t_loop:
    lea  si, DOBRO
    lea  di, TENT
    call copia
    lea  bx, TENT
    mov  al, A
    call empurra
    lea  si, TENT
    lea  di, PROD
    mov  al, A
    call mul_dig
    lea  dx, msgTab
```

```
    call texto
    lea  si, TENT
    xor  cx, cx
    call escreve_arr
    lea  dx, msgVez
    call texto
    mov  al, A
    call escreve_dig
    lea  dx, msgIg
    call texto
    lea  si, PROD
    xor  cx, cx
    call escreve_arr
    lea  si, PROD
    lea  di, RESTO
    call compara
    ja   t_pass
    lea  si, PROD
    lea  di, PRODOK
    call copia
    mov  al, A
    mov  MELHOR, al
    cmp  al, 9
    je   t_fim
    inc  al
    mov  A, al
    jmp  t_loop
t_pass:
    lea  dx, msgMai
    call texto
    lea  si, RESTO
    xor  cx, cx
    call escreve_arr
    lea  dx, msgUsa
    call texto
    mov  al, MELHOR
    call escreve_dig
t_fim:

    lea  dx, msgRes
    call texto
    lea  si, RESTO
    xor  cx, cx
    call escreve_arr
    lea  dx, msgMen
    call texto
    lea  si, PRODOK
    xor  cx, cx
    call escreve_arr
    lea  dx, msgIg
    call texto
    lea  si, RESTO
    lea  di, PRODOK
    call subtrai
    lea  si, RESTO
    xor  cx, cx
    call escreve_arr

    lea  dx, msgRz
    call texto
```

```
    lea  si, RAIZ
    xor  cx, cx
    call escreve_arr
    lea  dx, msgX10
    call texto
    mov  al, MELHOR
    call escreve_dig
    lea  dx, msgIg
    call texto
    lea  bx, RAIZ
    mov  al, MELHOR
    call empurra
    lea  si, RAIZ
    xor  cx, cx
    call escreve_arr
    mov  ax, IDX
    add  ax, 2
    mov  IDX, ax
    cmp  ax, NG
    jae  g_fim
    jmp  gciclo
g_fim:

    lea  dx, msgFim
    call texto
    lea  dx, msgRQ
    call texto
    lea  si, RAIZ
    mov  cx, NDECR
    call escreve_arr
    lea  dx, msgR
    call texto
    lea  si, RESTO
    xor  cx, cx
    call e_zero
    je   r_esc
    mov  cx, NDECR2
r_esc:
    call escreve_arr
    mov  ah, 4Ch
    int  21h

ler_num proc
    push ax
    push bx
    push cx
    push dx
    push si
    push di
    lea  bx, BUF
    xor  cx, cx
    xor  di, di
    xor  dx, dx
    mov  al, 0
    mov  SINAL, al
    mov  ah, 01h
    int  21h
    cmp  al, '-'
    jne  ln_ver
    mov  al, 1
```

```
    mov  SINAL, al
ln_prox:
    mov  ah, 01h
    int  21h
ln_ver:
    cmp  al, 13
    je   ln_fim
    cmp  al, ','
    je   ln_virg
    cmp  al, '.'
    je   ln_virg
    sub  al, '0'
    cmp  al, 9
    ja   ln_prox
    cmp  cx, MAXD
    jae  ln_mau
    mov  si, cx
    mov  [bx+si], al
    inc  cx
    cmp  dh, 0
    je   ln_prox
    inc  di
    jmp  ln_prox
ln_virg:
    cmp  dh, 0
    jne  ln_mau
    mov  dh, 1
    jmp  ln_prox
ln_mau:
    mov  dl, 1
    jmp  ln_prox
ln_fim:
    cmp  cx, 0
    je   ln_inv
    cmp  dl, 0
    jne  ln_inv
    mov  NDIG, cx
    mov  NDEC, di
    clc
    jmp  ln_sai
ln_inv:
    stc
ln_sai:
    pop  di
    pop  si
    pop  dx
    pop  cx
    pop  bx
    pop  ax
    ret
ler_num endp

para_array proc
    push ax
    push cx
    push si
    call zera
    xor  si, si
    mov  cx, NDIG
pa_loop:
    mov  al, BUF[si]
```

```
    call empurra
    inc  si
    loop pa_loop
    pop  si
    pop  cx
    pop  ax
    ret
para_array endp

zera proc
    push ax
    push cx
    push si
    xor  si, si
    xor  al, al
    mov  cx, LEN
z_loop:
    mov  [bx+si], al
    inc  si
    loop z_loop
    pop  si
    pop  cx
    pop  ax
    ret
zera endp

empurra proc
    push cx
    push dx
    push si
    mov  dl, al
    xor  si, si
    mov  cx, LEN-1
e_loop:
    mov  al, [bx+si+1]
    mov  [bx+si], al
    inc  si
    loop e_loop
    mov  [bx+si], dl
    mov  al, dl
    pop  si
    pop  dx
    pop  cx
    ret
empurra endp

copia proc
    push ax
    push bx
    push cx
    xor  bx, bx
    mov  cx, LEN
cp_loop:
    mov  al, [si+bx]
    mov  [di+bx], al
    inc  bx
    loop cp_loop
    pop  cx
    pop  bx
    pop  ax
```

```
    ret
copia endp

compara proc
    push ax
    push cx
    push si
    push di
    mov  cx, LEN
c_loop:
    mov  al, [si]
    cmp  al, [di]
    jne  c_fim
    inc  si
    inc  di
    loop c_loop
    cmp  al, al
c_fim:
    pop  di
    pop  si
    pop  cx
    pop  ax
    ret
compara endp

subtrai proc
    push ax
    push bx
    mov  bx, LEN-1
    xor  ah, ah
s_loop:
    mov  al, [si+bx]
    sub  al, ah
    sub  al, [di+bx]
    mov  ah, 0
    jns  s_ok
    add  al, 10
    mov  ah, 1
s_ok:
    mov  [si+bx], al
    dec  bx
    jns  s_loop
    pop  bx
    pop  ax
    ret
subtrai endp

mul_dig proc
    push ax
    push bx
    push dx
    mov  dl, al
    xor  dh, dh
    mov  bx, LEN-1
m_loop:
    mov  al, [si+bx]
    mul  dl
    add  al, dh
    xor  dh, dh
m_dez:
```

```
    cmp  al, 9
    jbe  m_un
    sub  al, 10
    inc  dh
    jmp  m_dez
m_un:
    mov  [di+bx], al
    dec  bx
    jns  m_loop
    pop  dx
    pop  bx
    pop  ax
    ret
mul_dig endp

e_zero proc
    push ax
    push bx
    push cx
    xor  al, al
    xor  bx, bx
    mov  cx, LEN
ez_loop:
    or   al, [si+bx]
    inc  bx
    loop ez_loop
    cmp  al, 0
    pop  cx
    pop  bx
    pop  ax
    ret
e_zero endp

escreve_arr proc
    push ax
    push bx
    push dx
    push di
    mov  di, LEN-1
    sub  di, cx
    xor  bx, bx
ea_proc:
    cmp  bx, di
    jae  ea_loop
    mov  al, [si+bx]
    cmp  al, 0
    jne  ea_loop
    inc  bx
    jmp  ea_proc
ea_loop:
    mov  dl, [si+bx]
    add  dl, '0'
    mov  ah, 02h
    int  21h
    cmp  bx, di
    jne  ea_sv
    cmp  cx, 0
    je   ea_sv
    mov  dl, ','
    mov  ah, 02h
```

```
    int  21h
ea_sv:
    inc  bx
    cmp  bx, LEN-1
    jbe  ea_loop
    pop  di
    pop  dx
    pop  bx
    pop  ax
    ret
escreve_arr endp

escreve_dig proc
    push ax
    push dx
    mov  dl, al
    add  dl, '0'
    mov  ah, 02h
    int  21h
    pop  dx
    pop  ax
    ret
escreve_dig endp

escreve_n8 proc
    push ax
    xor  ah, ah
n8_dez:
    cmp  al, 9
    jbe  n8_esc
    sub  al, 10
    inc  ah
    jmp  n8_dez
n8_esc:
    cmp  ah, 0
    je   n8_un
    push ax
    mov  al, ah
    call escreve_dig
    pop  ax
n8_un:
    call escreve_dig
    pop  ax
    ret
escreve_n8 endp

menos proc
    push ax
    push dx
    mov  dl, '-'
    mov  ah, 02h
    int  21h
    pop  dx
    pop  ax
    ret
menos endp

texto proc
    push ax
    mov  ah, 09h
```

```
    int  21h
    pop  ax
    ret
texto endp

end main
```